INTERNSHIP PROJECT REPORT

Submitted in partial fulfilment of the internship requirements at

Technology Innovation Hub, IIT Guwahati

FloodEye

Real-time Flood Alert & Water Level Monitoring System

'Every Drop Monitored, Every Life Protected."

Project Domain:	IoT, Machine Learning & Web Dashboard
Internship Organization:	Technology Innovation Hub (TIH), IIT Guwahati
Internship Duration:	*[Enter Start Date] – [Enter End Date]*
Institution:	*[Your College/University Name]*
Department:	*[Your Department Name]*
Guide / Mentor:	*[Mentor Name, Designation]*
Date:	July 2026

Team Members

Moharnab Gogoi	IoT Backend & Cloud Integration
Aryyaman Bora	Frontend & UI Engineering
Mayuree Khanikar	Documentation & Research
Indrani Gogoi	Documentation

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Flooding is one of the most frequent and destructive natural disasters affecting millions of people globally, and the Indian subcontinent — in particular the Brahmaputra river basin of Assam — is among the most vulnerable regions in the world. Each monsoon season, rapidly rising water levels inundate vast agricultural tracts, damage infrastructure, displace communities, and claim lives. Historically, disaster response in these regions has been reactive: by the time human observers detect an unsafe water level and communicate the warning to downstream communities, the available evacuation window is dangerously narrow.

The convergence of Internet of Things (IoT) sensor technology, real-time cloud data platforms, and modern machine learning has created a new class of early warning systems that can operate continuously, inexpensively, and at the network edge. Instead of relying on manual gauge readings or expensive government-operated stations, a low-cost ESP32 microcontroller equipped with environmental sensors can upload telemetry to the cloud every 15 seconds, 24 hours a day, feeding an intelligent dashboard that monitors conditions, generates alerts, and predicts water levels up to four hours in advance.

FloodEye is a response to this need: a full-stack Progressive Web Application (PWA) developed during the internship at the Technology Innovation Hub (TIH), IIT Guwahati, that brings together IoT hardware, cloud integration, a modern web frontend, and an in-browser machine learning model into a cohesive, deployable flood early warning system.

1.2 Problem Statement

Despite the widespread availability of affordable IoT components and cloud platforms, most rural and semi-urban monitoring deployments in flood-prone regions suffer from one or more of the following deficiencies:

- **Manual data collection:** Water level gauges require physical inspection, introducing delays and gaps.

- **No predictive capability:** Existing simple telemetry dashboards display only current readings; they cannot forecast whether levels will reach dangerous thresholds in the next several hours.
- **No offline resilience:** If the network link to a sensor breaks, many dashboards simply show an error, losing even the last known reading.
- **High infrastructure cost:** Dedicated server deployments for prediction models require significant maintenance overhead not feasible for research or municipal teams.
- **Poor user experience:** Many monitoring tools present raw JSON feeds or rudimentary tables rather than actionable, visually clear information.

FloodEye was designed to address every one of these gaps using modern, open, and cost-effective technology.

1.3 Existing Challenges

Several technical and operational challenges were encountered that shaped the design of FloodEye:

1. **Sensor data latency:** The ESP32 uploads to ThingSpeak every 15 seconds; the dashboard must cache data intelligently to avoid serving stale readings while preventing excessive API calls.
2. **Model deployment at the edge:** Running a deep learning model server-side requires dedicated infrastructure. Running it entirely in the browser using TensorFlow.js requires careful weight management and compatibility patching.
3. **Keras 3 – TensorFlow.js incompatibility:** The trained model uses Keras 3, whose export format differs from what TensorFlow.js expects, requiring a custom patch script.
4. **Insufficient real-time data for prediction:** The model requires 48 consecutive hourly readings; new installations do not have this history, requiring a sample-data padding strategy.
5. **Alert fatigue:** Naive threshold alerting fires continuously while a condition persists. A per-type cooldown and deduplication system was required.

1.4 Motivation

The motivation for FloodEye stems from three sources:

- **Social impact:** Assam experiences severe annual flooding. An affordable, deployable early warning system could give vulnerable communities the minutes to hours of advance notice needed to protect lives and property.
- **Technical exploration:** The project provided an opportunity to integrate four distinct engineering disciplines — embedded systems, cloud data pipelines, full-stack web development, and applied machine learning — into a single cohesive product.
- **Research alignment:** The internship host, TIH IIT Guwahati, focuses on technology translation from lab to field; FloodEye is a concrete demonstration of how academic research in IoT and ML can be packaged for real-world deployment.

1.5 Objectives

The primary objectives of the FloodEye project are:

1. Monitor water level and environmental conditions in real time using an ESP32 microcontroller equipped with BMP180, DHT11, and HC-SR04 sensors.
2. Transmit sensor readings to the ThingSpeak cloud platform at 15-second intervals via HTTP.
3. Build a responsive Next.js full-stack web application that fetches, caches, and visualises live and historical telemetry.
4. Implement a smart alerting engine with configurable thresholds and per-type cooldowns for: high water level, rapid water rise, high temperature, high humidity, rapid pressure change, and sensor offline conditions.
5. Deploy and run a 1D-CNN + Bidirectional LSTM + Attention deep learning model entirely within the browser using TensorFlow.js, providing 4-hour water level forecasts.
6. Implement a Progressive Web App (PWA) so users can install the dashboard on mobile and desktop devices and receive native OS notifications.
7. Deploy the entire application on Vercel with zero-server-management continuous deployment.

1.6 Scope of the Project

FloodEye covers the following scope:

- **In scope:** Single-channel ESP32 sensor node; ThingSpeak cloud integration; Next.js full-stack dashboard; browser-side ML inference; PWA; Vercel deployment; real-time alerts; historical analytics; PDF/CSV export; interactive map.
- **Out of scope:** Multi-node mesh networks (future enhancement); MQTT broker integration; on-device model inference on the ESP32; SMS/email notification gateway; user authentication with database persistence (authentication routes exist as a structural placeholder only).

1.7 Expected Outcomes

- A fully functional, publicly deployable flood monitoring dashboard accessible from any modern web browser.
- A trained deep learning model capable of predicting water levels 4 hours ahead with a mean absolute error below 5 cm on the test dataset.
- Automated alerts with configurable thresholds, cooldowns, and native browser push notification support.
- A Progressive Web App installable on Android, iOS, and desktop platforms.
- Comprehensive technical documentation suitable for replication, extension, and field deployment.

Chapter 2

Vision and Mission

2.1 Vision

> **"To leverage advanced IoT technologies, sensor networks and real-time data analytics for accurate flood and water level monitoring, enabling timely early warnings to protect lives, property and the environment."**
>
> *"Building a safer society and a flood-resilient future through smart technology."*

The vision of FloodEye is rooted in the conviction that the catastrophic human and economic cost of flooding in vulnerable regions like Assam is not inevitable. Floods cannot be prevented, but the damage they inflict on unprepared communities can be dramatically reduced when accurate, real-time information is placed in the hands of those who need it most — residents, local administrators, disaster management authorities, and first responders.

2.1.1 Expanding the Vision

The long-term vision extends beyond a single sensor node dashboard to a distributed, intelligent flood early warning network:

- **Citywide and basin-wide coverage:** Multiple FloodEye nodes deployed along riverbanks, drainage canals, and flood plains, each uploading independently to ThingSpeak channels and visualised on a unified regional map.
- **Democratised access:** A PWA architecture ensures that any resident with a smartphone and a data connection can receive flood alerts and monitor local conditions without installing a native app.
- **Environmental intelligence:** Beyond water level, the simultaneous monitoring of temperature, humidity, pressure, and altitude enables multi-variable environmental

analysis and the detection of weather patterns that correlate with elevated flood risk.

- **AI-driven forecasting at scale:** As more sensor data accumulates, the machine learning model can be retrained to improve predictive accuracy and extended to cover longer forecast horizons.

2.2 Mission

The operational mission of FloodEye, as stated in the project charter, is:

1. **Monitor water levels in real time using IoT-enabled sensors.**
   The ESP32 microcontroller, equipped with the HC-SR04 ultrasonic distance sensor, measures water proximity at 15-second intervals and uploads readings continuously via HTTP to the ThingSpeak cloud platform.

2. **Provide timely and reliable flood early warning alerts.**
   The smart alerting engine evaluates each incoming reading against configurable thresholds for water level, rate of rise, pressure drop, temperature, and humidity. Alerts are classified as *critical*, *warning*, or *info* and are pushed to the user as in-app notifications and, where permission is granted, as native OS-level push notifications.

3. **Deliver accurate and trustworthy data to government agencies and citizens.**
   The dashboard presents data through interactive, time-series charts with configurable historical windows (1 hour, 6 hours, 24 hours, 7 days, 30 days) and provides CSV and PDF export functionality so that data can be shared with authorities and used in official reports.

4. **Reduce disaster risks and improve community preparedness.**
   By predicting water levels 4 hours in advance using the embedded CNN-BiLSTM-Attention model, FloodEye gives communities a meaningful advance warning window to prepare for potential flooding, move property to safety, and initiate evacuation if required.

2.3 Long-Term Goals

Table 2.1: FloodEye Long-Term Strategic Goals

Goal	Description
Multi-Node Network	Deploy sensor nodes across multiple locations along rivers/canals with a unified map view
MQTT Integration	Replace HTTP polling with MQTT-over-WebSocket for lower-latency IoT telemetry
Improved ML Models	Retrain with larger datasets; incorporate weather API forecasts as additional features
Mobile App	Native iOS/Android companion app with background push notification support
Authority Integration	API integration with state disaster management portals for automated alerts
Community Dashboard	Public-facing read-only view for residents and media without authentication

2.4 Real-World Impact

FloodEye has been designed with direct applicability to the Brahmaputra Valley flood plains of Assam, where annual monsoon flooding affects millions of people. The system's key impact areas are:

- **Life safety:** A 4-hour advance warning of dangerous water levels gives households sufficient time to move to higher ground and alert neighbours.
- **Property protection:** Early warnings enable the timely removal of vehicles, livestock, and stored goods from flood-vulnerable areas.
- **Agricultural loss reduction:** Farmers can harvest or protect crops when imminent flooding is forecast.
- **Cost-effective deployment:** The entire hardware component — ESP32, BMP180, DHT11, HC-SR04 — costs under €15, making wide-scale deployment economically viable.

2.5 Scalability and Social Benefits

The FloodEye architecture is horizontally scalable by design:

- **Vercel deployment** provides automatic serverless scaling; traffic spikes during flood events do not degrade dashboard performance.
- **ThingSpeak channels** can be added independently for each sensor node with no change to the backend code.
- **The PWA** functions as an installable app on any device, eliminating the need for app store submissions or native development for initial deployment.
- **Browser-side ML inference** removes the need for a GPU-equipped server; the prediction runs on the user's own device, reducing hosting cost to zero for the ML component.

Chapter 3

Project Overview

3.1 System Description

FloodEye is a highly responsive Progressive Web Application (PWA) built with Next.js 16 (App Router) and Tailwind CSS v4. It is designed to visualise real-time water level data and environmental conditions captured by an ESP32 microcontroller. The system combines a glassmorphism UI, interactive Recharts-powered charts, an intelligent alerting engine, a live MapLibre GL map, and an in-browser TensorFlow.js deep learning model into a unified monitoring and prediction platform.

The system operates in three interconnected layers:

1. **Hardware Layer:** An ESP32 DevKit with BMP180 (pressure, altitude), DHT11 (temperature, humidity), and HC-SR04 (water distance) sensors uploads readings to ThingSpeak every 15 seconds.

2. **Backend Layer:** Next.js Route Handlers fetch data from ThingSpeak, apply caching with request coalescing, compute analytics statistics, and serve clean telemetry objects to the frontend.

3. **Frontend Layer:** A React-based dashboard renders real-time sensor cards, historical trend charts, an alert log, a device location map, an ML prediction card, and an environmental comfort score.

3.2 Core Modules

Table 3.1: FloodEye Core Modules

Module	Description
IoT Hardware	ESP32 + sensors; uploads telemetry every 15 s via HTTP POST
ThingSpeak Integration	Cloud IoT data platform; stores field-mapped sensor readings
TelemetryProvider	React Context providing live data, history, alerts, settings, and device status to all dashboard components
Sensor Data APIs	Next.js Route Handlers: `/api/sensors`, `/api/history`, `/api/analytics`, `/api/environment-score`
Smart Alert Engine	Configurable threshold evaluation with cooldown and deduplication; native OS push notification support
ML Prediction Engine	1D-CNN + BiLSTM + Attention model; runs client-side in TensorFlow.js; predicts water level 4 hours ahead
Historical Analytics	Time-series charts for all five sensor channels with selectable windows: 1h, 6h, 24h, 7d, 30d
Interactive Map	MapLibre GL + MapTiler; shows sensor node location with live reading overlay
PWA	Installable offline-capable app via `next-pwa`; service worker caching; native install prompt
Data Export	CSV and PDF download of the last 100 historical readings using jsPDF and PapaParse

3.3 Key Features

- **AI-Powered Flood Prediction:** A hybrid 1D-CNN + Bidirectional LSTM + Attention model, trained on Assam sensor data and converted to TensorFlow.js, runs entirely in the browser and predicts water levels 4 hours in advance.
- **Progressive Web App (PWA):** Installable on iOS, Android, and desktop with custom install prompts and offline caching of the last known sensor readings.
- **Scrollytelling Landing Page:** An immersive introduction page built with GSAP ScrollTrigger and Lenis smooth scrolling, featuring animated sections and a background slideshow.
- **Live Interactive Map:** MapLibre GL and MapTiler render an interactive map

pinpointing the sensor node location with live water level, temperature, and humidity overlays.

- **Real-Time Telemetry:** Five sensor channels (Water Level, Temperature, Humidity, Atmospheric Pressure, Altitude) are displayed with live trend arrows, delta indicators, and sparkline mini-charts.
- **Dynamic Analytics:** Interactive line charts with configurable time windows and per-sensor colour coding.
- **Environmental Comfort Score:** A composite score derived from temperature and humidity readings, displayed as a percentage with a qualitative label (Excellent / Good / Moderate / Poor).
- **Smart Alerts Engine:** Seven alert types with severity levels (critical / warning / info), per-type cooldowns, deduplication, and native OS push notification support.
- **Offline/Cached Data:** When the ESP32 goes offline, the dashboard persists the last known reading in `localStorage` and displays an offline banner rather than an error.
- **Data Export:** History page supports one-click export of sensor logs as a CSV file or a formatted PDF report.
- **Role-Based Access:** Admin role unlocks the Settings page and diagnostic panels; the User role provides read-only access.

3.4 Functional Workflow

The end-to-end functional workflow of FloodEye is illustrated in Figure 3.1.

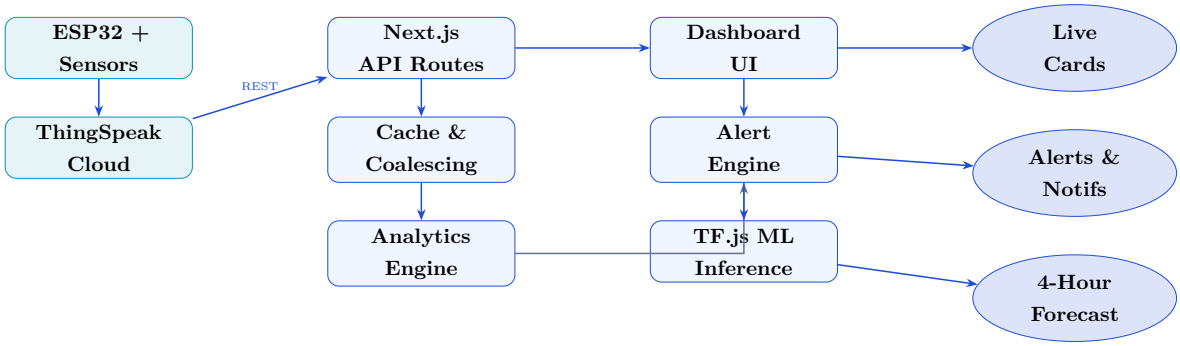


Figure 3.1: FloodEye End-to-End Functional Pipeline

3.5 User Workflow

A typical user interaction with FloodEye follows this sequence:

1. The user navigates to the FloodEye URL (e.g. on Vercel) or opens the installed PWA on their device.
2. The immersive landing page loads with GSAP animations and a background slideshow of flood monitoring imagery.
3. The user clicks "Go to Dashboard" or the direct dashboard link.
4. The dashboard fetches the latest sensor reading from `/api/sensors`. The TelemetryProvider begins polling at the configured refresh interval (default: 15 seconds).
5. Five sensor cards display live values (Water Level, Temperature, Humidity, Pressure, Altitude) with trend arrows.
6. Simultaneously, the `useFloodPrediction` hook loads the TensorFlow.js model from `/public/tfjs_model/` and, once the model is ready and at least one reading is available (padded with sample data if necessary), runs inference to display the 4-hour water level forecast.
7. If any sensor reading breaches a configured threshold, an alert card appears in the Alert Panel. Critical alerts trigger a native browser push notification (if permission was granted).
8. The user can navigate to the **Analytics** page to view per-channel time-series charts with the time-window selector.
9. The **History** page presents a statistical summary table and allows download of sensor logs as CSV or PDF.
10. The **Predictions** page provides a detailed ML forecasting panel with risk gauge and feature contributions.
11. The **Alerts** page shows the full alert log, filterable by severity.
12. Admin users can navigate to **Settings** to adjust alert thresholds, refresh interval, and toggle simulation mode.

3.6 Navigation Structure

FloodEye has two main navigation zones:

- **Landing Zone:** Root route (`/`) — the scrollytelling landing page with navigation bar, hero section, feature showcase, team profiles, and technology section.
- **Dashboard Zone:** All routes under `/dashboard` — protected by a shared layout with a sidebar navigation containing: Dashboard, Analytics, History, Predictions, Alerts, Devices, and Settings.

Chapter 4

System Architecture

4.1 Architectural Overview

FloodEye follows a three-tier architecture consisting of an IoT hardware layer, a cloud-connected backend layer, and a client-side frontend layer. All tiers are orchestrated through the Next.js full-stack framework, which unifies the backend API routes and the React frontend into a single deployable unit. Figure 4.1 illustrates the overall architecture.

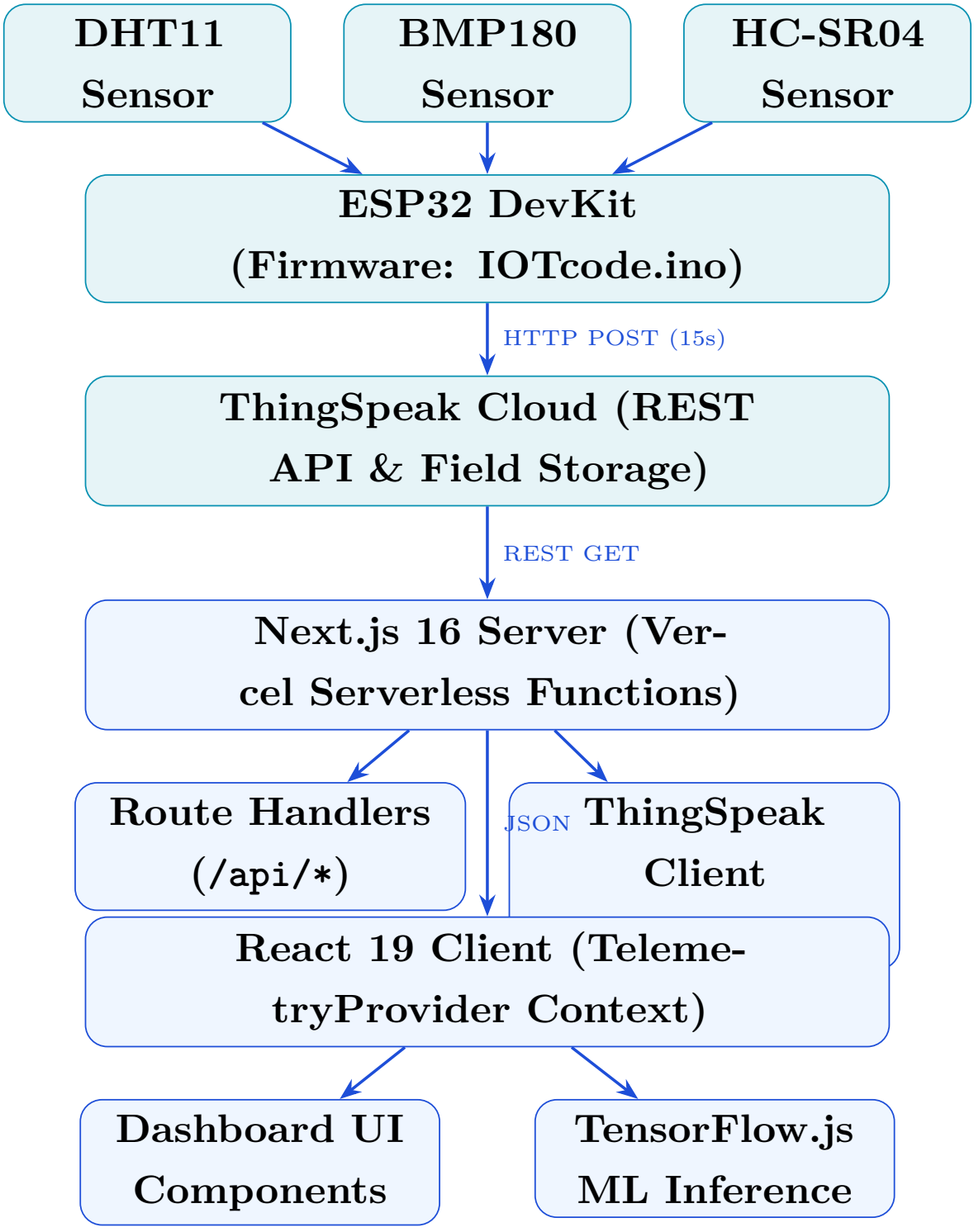


Figure 4.1: FloodEye Three-Tier System Architecture

4.2 IoT Hardware Layer

The hardware layer consists of an ESP32 DevKit microcontroller connected to three sensors:

- **BMP180 (I^2C):** Measures atmospheric pressure (hPa) and derives altitude (m).

Connected via I^2C bus on GPIO 21 (SDA) and GPIO 22 (SCL).

- **DHT11 (Digital):** Measures ambient temperature (°C) and relative humidity (%). A simple single-wire digital sensor.
- **HC-SR04 (GPIO):** Ultrasonic distance sensor. The distance reading, in centimetres, represents the proximity to the water surface (inversely proportional to water level).

The firmware (`IOTcode/IOTcode.ino`) initialises the BMP180 via the `Adafruit_BMP085` library and polls readings in a 2-second loop, printing values to the serial monitor. The full deployment extends this sketch to post field values to ThingSpeak via an HTTP client library.

4.3 ThingSpeak Cloud Layer

ThingSpeak serves as the cloud IoT data platform. The ESP32 uploads to a ThingSpeak channel, whose five fields are mapped as shown in Table 4.1.

Table 4.1: ThingSpeak Channel Field Mapping

ThingSpeak Field	Sensor	Unit
Field 1	Temperature (DHT11)	°C
Field 2	Humidity (DHT11)	%
Field 3	Atmospheric Pressure (BMP180)	hPa
Field 4	Altitude (BMP180)	m
Field 5	Water Distance (HC-SR04)	cm

ThingSpeak imposes a free-tier rate limit of one update per 15 seconds, which aligns with the ESP32 firmware upload interval. The REST API supports both "last entry" and "range" queries used by the Next.js backend.

4.4 Backend Layer

The backend is implemented as Next.js Route Handlers (the App Router equivalent of API routes). All server logic resides in the `app/api/` directory. The key components are:

- **`lib/thingspeak.ts`:** A server-side ThingSpeak client with an in-memory cache and request coalescing. The latest-reading cache has a 14-second TTL (just under the 15-second upload interval); the history cache has a 55-second TTL. Request coalescing prevents cache stampedes when multiple server renders arrive simultaneously.

- **Route Handlers:** `/api/sensors`, `/api/history`, `/api/analytics`, `/api/environment-score`, `/api/flood-predict` (placeholder for server-side inference). Each returns a structured JSON response.
- **Incremental Static Revalidation (ISR):** The `revalidate` export constant on each route sets the Next.js CDN cache TTL (15 s for sensors, 60 s for history/analytics).

4.5 Frontend Layer

The frontend is a React 19 application built with Next.js App Router. The central state management mechanism is the **TelemetryProvider**, a React Context provider that:

- Polls `/api/sensors` at a configurable interval (default 15 s).
- Persists the latest reading and history in `localStorage` for offline resilience.
- Evaluates alerts on every new data point using the `evaluateAlerts()` function.
- Exposes telemetry data, history array, alert list, device status, and settings to all child components via the `useTelemetry()` hook.

The ML inference layer runs separately in the **`useFloodPrediction`** hook, which loads the TensorFlow.js model from `/public/tfjs_model/`, engineers the 11 input features from the history array, and calls `predictFloodRisk()` to obtain a 4-hour water level forecast.

4.6 Data Flow

The complete data flow from sensor reading to dashboard visualisation is:

1. ESP32 reads sensor values and HTTP-POSTs them to ThingSpeak (every 15 s).
2. ThingSpeak stores the reading in its channel database.
3. The Next.js server's `TelemetryProvider` polls `/api/sensors` (every 15 s).
4. `/api/sensors` calls `getLatestReading()` in `lib/thingspeak.ts`, which fetches `https://api.thingspeak.com/channels/{id}/feeds/last.json` and parses the five field values into a typed `TelemetryData` object.
5. The result is cached for 14 s and returned as JSON to the frontend.

6. The TelemetryProvider stores the new reading in state and history, evaluates alerts, and persists to `localStorage`.
7. All dashboard components subscribed to `useTelemetry()` re-render with fresh data.
8. Concurrently, the `useFloodPrediction` hook detects a new history entry and schedules an ML inference run (debounced by 1 s).
9. The inference result updates the FloodPredictionCard and FloodPredictionPanel with the new forecast.

4.7 Cloud and Deployment Architecture

FloodEye is deployed on Vercel using its Git-integrated continuous deployment pipeline. Each push to the main branch triggers an automatic build and deployment. Key architectural features of the deployment:

- **Serverless Edge Functions:** Next.js Route Handlers are deployed as Vercel serverless functions, scaling automatically to handle traffic spikes.
- **CDN Caching:** ISR cache headers ensure that the ThingSpeak API is not hammered during high-traffic periods; cached responses are served from Vercel's global CDN edge network.
- **Environment Variables:** ThingSpeak credentials (`THINGSPEAK_CHANNEL_ID`, `THINGSPEAK_READ_`
and the MapTiler key (`NEXT_PUBLIC_MAPTILER_KEY`) are injected as Vercel environment variables, never exposed in the repository.
- **PWA Service Worker:** The `next-pwa` package generates a Workbox service worker that caches static assets for offline access.

Chapter 5

Technology Stack

This chapter documents every technology used in FloodEye, explaining the purpose of each component and the rationale for its selection.

5.1 Frontend Technologies

5.1.1 Next.js 16 (App Router)

Purpose: Full-stack React framework providing server-side rendering, API routes, file-system-based routing, and seamless Vercel deployment.
Why chosen: Next.js App Router colocates server components, client components, and API handlers in a single project. Its ISR (Incremental Static Regeneration) with the `revalidate` export allows fine-grained cache control per API route without a separate caching layer. The Vercel deployment integration is zero-configuration.
Version: 16.2.10

5.1.2 React 19

Purpose: UI component library; the foundation of all dashboard views.
Why chosen: React 19 introduces concurrent features and improved suspense semantics. The Context API (`TelemetryProvider`) serves as lightweight global state management without requiring Redux or Zustand.

5.1.3 TypeScript 5

Purpose: Static typing for all source files (`.ts`, `.tsx`).
Why chosen: TypeScript eliminates entire categories of runtime errors in the type-rich data pipeline (ThingSpeak field parsing, TelemetryData objects, alert severity enums). The `FloodPredictionResult`, `TelemetryData`, `DashboardSettings`, and `Alert` interfaces are all strongly typed.

5.1.4 Tailwind CSS v4

Purpose: Utility-first CSS framework for all component styling.
Why chosen: Tailwind v4's JIT (just-in-time) compiler generates minimal CSS bundles. The glassmorphism design language (`bg-white/40 backdrop-blur-xl border-white/60`) is cleanly expressed with utility classes without custom CSS files.

5.1.5 shadcn/ui

Purpose: Accessible, unstyled component primitives for buttons, dialogs, and form elements.
Why chosen: shadcn/ui components are installed directly into the project source rather than as an opaque dependency, making them fully customisable.

5.1.6 Recharts 3

Purpose: SVG-based charting library for all time-series line charts.
Why chosen: Recharts integrates naturally with React's rendering model. It provides `ResponsiveContainer`, `LineChart`, `Tooltip`, and `CartesianGrid` out-of-the-box, which are sufficient for FloodEye's chart requirements without the configuration overhead of D3.

5.1.7 MapLibre GL JS 5 + MapTiler

Purpose: WebGL-accelerated interactive map rendering.
Why chosen: MapLibre GL is an open-source fork of Mapbox GL with no usage-based pricing caps. MapTiler provides high-quality raster and vector tile services compatible with MapLibre's style spec. The `react-map-gl` wrapper provides React-native integration.

5.1.8 Framer Motion 12

Purpose: Animation library for UI micro-animations, card hover effects, and page transitions.
Why chosen: Framer Motion provides a declarative API (`motion.div`, `AnimatePresence`) that integrates directly with React's component tree.

5.1.9 GSAP 3 + @gsap/react

Purpose: ScrollTrigger-based animations on the landing page.
Why chosen: GSAP is the industry standard for high-performance scroll-driven

animations. The `ScrollytellingSections` component uses GSAP's `fromTo` and `ScrollTrigger` to choreograph section entrance animations.

5.1.10 Lenis 1.3

Purpose: Smooth, physics-based scroll inertia on the landing page.
Why chosen: Lenis provides buttery-smooth native scroll override that pairs naturally with GSAP ScrollTrigger.

5.1.11 Lucide React

Purpose: Icon library providing `Thermometer`, `Droplets`, `Wind`, `Mountain`, `Ruler`, `WifiOff`, `AlertTriangle`, and other icons used throughout the dashboard.

5.1.12 jsPDF + jspdf-autotable

Purpose: Client-side PDF generation for the sensor history export on the History page.
Why chosen: No server is required; the PDF is generated entirely in the browser and downloaded directly.

5.2 Machine Learning Technologies

5.2.1 TensorFlow 2 / Keras (Training)

Purpose: Python deep learning framework used to define, train, and export the flood prediction model.
Role: The model is defined in `Model/nb_code.py` (extracted from `FloodMonitoring.ipynb`). Training runs in Google Colab with GPU acceleration.

5.2.2 TensorFlow.js 4.22 (Inference)

Purpose: JavaScript port of TensorFlow; runs the trained model in the browser using WebGL acceleration.
Why chosen: Client-side inference eliminates the need for a dedicated ML serving infrastructure. The model runs on the user's device with no server cost for inference.
Integration: The custom `AttentionLayer` is reimplemented in TypeScript and registered with `tf.serialization.registerClass()` before model loading.

5.2.3 scikit-learn StandardScaler

Purpose: Feature normalisation during training.
Role: Scaler parameters (mean and scale vectors) are exported to `scalers.json` and loaded by the browser at inference time to apply the same normalisation.

5.2.4 TensorFlow.js Converter

Purpose: Converts the trained Keras `.h5` model to TensorFlow.js LayersModel format (`model.json` + `group1-shard1of1.bin`).

5.3 Backend Technologies

5.3.1 Next.js Route Handlers

Purpose: Serverless API endpoints at `app/api/*/route.ts`.

5.3.2 ThingSpeak REST API

Purpose: IoT cloud data platform; stores and serves field-mapped sensor readings.

5.4 IoT Technologies

5.4.1 ESP32 DevKit

Purpose: Main microcontroller; reads sensors and uploads data to ThingSpeak via Wi-Fi.
Framework: Arduino (C++ sketch: `IOTcode/IOTcode.ino`).

5.4.2 Adafruit BMP085 Library

Purpose: Arduino driver for the BMP180 pressure and temperature sensor.

5.4.3 DHT Sensor Library

Purpose: Arduino driver for the DHT11 temperature and humidity sensor.

5.5 Development Tools

Table 5.1: Development Tools and Versions

Tool	Version	Purpose
Node.js	v20.x	JavaScript runtime
npm	v10.x	Package manager
TypeScript	5.x	Static type checking
ESLint	9.x	Code quality linting
PostCSS	4.x	Tailwind CSS processing
Google Colab	—	GPU-accelerated ML training environment
Arduino IDE	—	ESP32 firmware development and flashing

5.6 Hosting and Infrastructure

Table 5.2: Hosting and Infrastructure

Service	Role
Vercel	Next.js application hosting; serverless functions; CDN; CI/CD pipeline
ThingSpeak	IoT cloud data storage and REST API
MapTiler	Vector tile service for MapLibre GL interactive map

Chapter 6

Frontend Implementation

The FloodEye frontend is a Next.js 16 App Router application. All pages live under either the root `/` (landing) or the nested `/dashboard/` route group. This chapter describes each page in detail.

6.1 Landing Page (`app/page.tsx`)

6.1.1 Purpose

The landing page serves as the public-facing introduction to FloodEye. It is designed to be visually striking and informative, communicating the project's purpose to new visitors before they enter the monitoring dashboard.

6.1.2 Components

- **LandingNav:** Fixed top navigation bar with logo, navigation links, and a "Go to Dashboard" call-to-action button. Includes blur-glass styling.
- **BackgroundSlideshow:** An auto-advancing slideshow of flood monitoring imagery rendered as a full-viewport background layer.
- **ScrollytellingSections:** The primary content component. Uses GSAP ScrollTrigger to animate content sections as the user scrolls: Hero, Problem Statement, Features Showcase, Technology Stack, Team Profiles (with individual member photos from `/public/`), and a call-to-action.
- **AssamNodeMap:** A landing-page specific map component showing the geographic coverage area of the monitoring network in Assam.
- **PWAInstallPrompt:** A floating prompt that appears when the browser's before-installprompt event fires, inviting the user to install the PWA.

6.1.3 Key Features

- Lenis smooth scroll integration for physics-based scroll inertia.

- GSAP `fromTo` and `ScrollTrigger` animations for each section entrance.
- Fully responsive layout adapting from mobile through desktop breakpoints.
- Team member profile cards with photos from the public directory.

6.2 Dashboard Home (`app/dashboard/page.tsx`)

6.2.1 Purpose

The main dashboard provides a comprehensive real-time overview of all sensor readings, device status, live charts, alerts, and the flood risk prediction in a single scrollable view.

6.2.2 Layout Structure

The dashboard is organised in four rows:

1. **Row 1:** Five sensor cards (Temperature, Humidity, Pressure, Altitude, Water Level) in a responsive grid (1 col on mobile → 5 cols on XL screens).
2. **Row 2:** A 2:1 grid containing the Water Level Trend chart (left, spanning 2 columns) and a right column with ComfortScoreCard, FloodPredictionCard, and DeviceStatusCard stacked vertically.
3. **Row 3:** The AlertPanel spanning the full width.
4. **Row 4:** The interactive sensor location map (MapLibre GL, loaded dynamically with SSR disabled).

6.2.3 Sensor Cards (SensorCard Component)

Each SensorCard renders:

- Current reading with unit.
- Trend arrow icon (up / down / stable) calculated by comparing the current reading against the previous one.
- Delta value showing the numeric change from the previous reading.
- A sparkline mini-chart rendered using the `historyData` prop.
- A status badge (normal / warning / critical) with colour coding.
- Offline-state: Cards are labelled "(Cached)" when the device is offline.

Alert thresholds per card:

- Temperature > 35°C: warning status.
- Humidity > 80%: warning status.
- Water Level < 20 cm: critical; < 40 cm: warning.

6.2.4 Water Level Trend Chart

An interactive Recharts `CustomLineChart` displays the last N data points of water level (cm) vs. time. A "Live" or "Cached" pill badge indicates the current data freshness.

6.2.5 Comfort Score Card

Calculates and displays the Environmental Comfort Score as a percentage derived from temperature and humidity. Score ranges map to qualitative labels (Excellent / Good / Moderate / Poor).

6.2.6 Flood Prediction Card

A compact summary of the ML prediction result: predicted water level in 4 hours, risk level badge (Low / Moderate / High / Critical), and flood probability percentage.

6.2.7 Device Status Card

Displays ESP32 online/offline status, ThingSpeak connection status, and last upload time derived from the `DeviceStatus` object.

6.2.8 Alert Panel

The AlertPanel component renders a horizontally scrollable list of active alerts, with severity-coded colour bands. It links to the full Alerts page.

6.2.9 Offline Behaviour

When the ESP32 is unreachable:

- An orange **OfflineBanner** is shown at the top with the time of the last live reading.
- Sensor cards show "(Cached)" labels.
- The Live badge on the chart changes to "Cached".
- A **NoDataState** component replaces the dashboard if no cached data exists at all.

6.3 Analytics Page (`app/dashboard/analytics/page.tsx`)

6.3.1 Purpose

Provides individual time-series charts for all five sensor channels, each rendered in its own card.

6.3.2 Components

- **TimeFilter:** A pill-tab selector for 1h, 6h, 24h, 7d, 30d windows. Selecting a window triggers a new `/api/history?range={window}` request.
- **Five CustomLineChart cards:** Temperature (°C), Humidity (%), Atmospheric Pressure (hPa), Altitude (m), Water Level (cm). The Water Level card uses a dark gradient background to visually distinguish it as the primary safety metric.
- The Pressure chart spans two columns (`lg:col-span-2`) as a wide chart for better readability.

6.4 History Page (`app/dashboard/history/page.tsx`)

6.4.1 Purpose

Displays statistical summaries of sensor history and provides data export functionality.

6.4.2 Components

- **StatCard grid:** Six cards showing average, min, and max for temperature, humidity, and water level across the current history window.
- **Data table:** A scrollable table of the last 100 readings with columns: Time, Temperature, Humidity, Pressure, Altitude, Water Level.
- **Export buttons:**
  - **CSV Export:** Generates a Blob from the history array and downloads it as `floodeye_logs_{date}.csv`.
  - **PDF Export:** Uses jsPDF + jspdf-autotable to generate a formatted PDF with the FloodEye header and a data table.

6.5 Predictions Page (`app/dashboard/predictions/page.tsx`)

6.5.1 Purpose

A dedicated full-page view of the ML flood forecasting results with admin diagnostic tools.

6.5.2 Components

- **FloodPredictionPanel:** A large card displaying the predicted water level, a circular risk gauge, flood probability bar, confidence score, feature contributions list (Water Level, Temperature, Humidity, Pressure), and a timestamp of the last inference.
- **Admin Diagnostics (admin role only):**
  - Model Architecture card: Type (CNN-BiLSTM-Att), Input Shape [48, 11], Output (4hr Forecast), MAE (Train: 3.42 cm).
  - Inference Engine card: Provider (TensorFlow.js/WebGL), live status (COMPUTING/READY), last run timestamp.
  - Settings CTA card: Links to the Settings page.

6.6 Alerts Page (`app/dashboard/alerts/page.tsx`)

6.6.1 Purpose

The full alert log with severity filtering and per-alert dismissal.

6.6.2 Components

- Summary badges showing counts of Critical, Warning, and Info alerts.
- **AlertCard:** Each alert card displays: severity icon, type label, severity badge, timestamp (relative, e.g. "3m ago"), message text, and a dismiss button.
- Alerts are ordered critical-first, then warning, then info.
- "Clear All" button clears the log and resets all alert cooldown timers.

Alert types implemented:

- HIGH_TEMPERATURE, HIGH_HUMIDITY, RAPID_PRESSURE_CHANGE

- HIGH_WATER_LEVEL, RAPID_WATER_RISE, OBJECT_TOO_CLOSE, SENSOR_OFFLINE

6.7 Settings Page (`app/dashboard/settings/page.tsx`)

6.7.1 Purpose

Allows admin users to configure alert thresholds, refresh interval, and simulation mode.

6.7.2 Access Control

The Settings page checks `userRole === "admin"` from the TelemetryProvider context. Non-admin users see an access-restricted screen with a ShieldAlert icon.

6.7.3 Settings Sections

- **Connection Settings:** Refresh interval selector (5s, 15s, 30s, 60s). Simulation mode toggle (uses synthetic data instead of live ThingSpeak readings).
- **Alert Thresholds:** Input fields for Temperature threshold (°C), Humidity threshold (%), Water Level threshold (cm), Pressure threshold (hPa).
- **Save:** Persists settings to the TelemetryProvider context (in-memory; not yet persisted to a database in the current implementation).

6.8 Devices Page (`app/dashboard/devices/`)

The Devices section provides device management views showing registered ESP32 nodes and their connection status. In the current implementation, this page displays the status of the single configured ThingSpeak channel.

6.9 Responsive Design

All dashboard pages are fully responsive using Tailwind CSS breakpoint utilities:

- **Mobile (default):** Single column layout; cards stack vertically.
- **Tablet (`sm:`):** Sensor cards 2-column grid.
- **Laptop (`lg:`):** 3-column grid for sensor cards; 3:1 chart+sidebar split.
- **Desktop (`xl:`):** 5-column sensor card grid; full analytics 2-column layout.

Chapter 7

Application Navigation

7.1 Navigation Architecture

FloodEye uses Next.js App Router's file-system-based routing. The application has two primary zones: the public Landing Zone and the authenticated Dashboard Zone. The dashboard zone uses a shared layout (`app/dashboard/layout.tsx`) that renders the sidebar navigation on every dashboard page.

7.2 Navigation Flow Diagram

Figure 7.1 illustrates the complete navigation flow of the application.

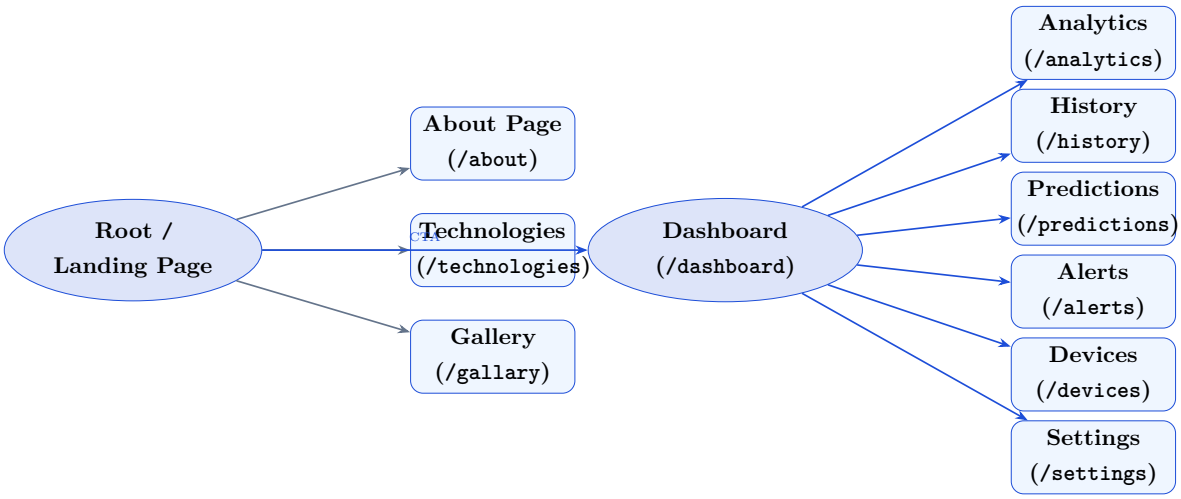


Figure 7.1: FloodEye Complete Navigation Flow

7.3 Landing Zone Pages

Table 7.1: Landing Zone Pages

Route	Page	Purpose
`/`	Landing Page	Scrollytelling intro with GSAP animations, feature showcase, team section
`/about`	About	Project background, team details, internship context
`/technologies`	Technologies	Technology stack showcase with animated cards
`/gallary`	Gallery	Project photo gallery and hardware demonstration images

7.4 Dashboard Zone Layout

The dashboard zone uses a shared `layout.tsx` that wraps all `/dashboard/*` pages. This layout renders:

- **Sidebar Navigation:** A fixed left sidebar containing:
  - FloodEye logo and version.
  - Navigation links: Dashboard, Analytics, History, Predictions, Alerts, Devices, Settings.
  - User role indicator (Admin / User) and username display.
  - Alert bell icon with unread count badge.
- **Top Header Bar:** Displays the current page title, date/time, connection status indicator, and the PWA install button.
- **Main Content Area:** The `{children}` slot where each page renders.
- **TelemetryProvider:** The layout wraps all content in the TelemetryProvider context so every dashboard page has access to live sensor data.

7.5 Dashboard Pages

Table 7.2: Dashboard Zone Pages and Features

Route	Features
`/dashboard`	Live sensor cards, water level chart, comfort score, flood prediction card, alert panel, device map
`/dashboard/analytics`	Five individual time-series charts, time window selector (1h–30d)
`/dashboard/history`	Statistical summary cards, reading table, CSV export, PDF export
`/dashboard/predictions`	ML prediction panel, risk gauge, feature contributions, admin diagnostics
`/dashboard/alerts`	Full alert log, severity badges, dismiss and clear functionality
`/dashboard/devices`	Device registration and status (single-channel in current implementation)
`/dashboard/settings`	Threshold configuration, refresh interval, simulation toggle (admin only)

7.6 Authentication Routes

The `app/(auth)/` route group contains structural placeholder pages for a future authentication system. In the current implementation, user role is stored in `localStorage` (key: `floodeye_session`) and toggled through the Settings page, rather than through a proper login/password flow. This is noted as a known limitation and future enhancement.

7.7 PWA Navigation

When FloodEye is installed as a PWA:

- The app opens in a standalone browser window (without browser chrome) displaying the dashboard home.
- The service worker (`public/sw.js`) caches static assets and serves the last known dashboard state offline.
- Native back/forward navigation works as in a native app.
- The `PWAInstallButton` in the top header bar triggers the native install prompt when the PWA is not yet installed.

Chapter 8

Backend Implementation

8.1 Overview

The FloodEye backend is implemented using Next.js App Router Route Handlers, which run as serverless functions on Vercel. There is no separate Express.js or FastAPI server. All backend logic is colocated with the frontend in the single Next.js project.

8.2 Folder Structure

```
app/
  api/
    sensors/           route.ts    -- GET /api/sensors
    history/           route.ts    -- GET /api/history
    analytics/         route.ts    -- GET /api/analytics
    environment-score/ route.ts    -- GET /api/environment-score
    flood-predict/     route.ts    -- POST /api/flood-predict
lib/
  thingspeak.ts        -- ThingSpeak client + cache
  flood-model.ts       -- TF.js model loader + inference
  utils.ts             -- cn() utility
  actions/             -- Next.js Server Actions (if any)
hooks/
  useFloodPrediction.ts  -- Client-side ML hook
  useNativeNotification.ts
  use-outside-click.ts
components/
  providers/
    TelemetryProvider.tsx  -- Central data/state provider
```

Listing 8.1: Backend-relevant directory structure

8.3 ThingSpeak Client (`lib/thingspeak.ts`)

The ThingSpeak client is the core of the backend data layer. It encapsulates all communication with the ThingSpeak REST API and implements a sophisticated

caching strategy to minimise external API calls.

8.3.1 Field Parsing

The `feedToTelemetry()` function maps ThingSpeak feed fields to the `TelemetryData` type:

```
function feedToTelemetry(feed: ThingSpeakFeed): TelemetryData {
  return {
    temperature: parseField(feed.field1),      // DHT11 temp (deg C)
    humidity:    parseField(feed.field2),      // DHT11 humidity (%)
    pressure:    parseField(feed.field3, 1013.25), // BMP180 (hPa)
    altitude:    parseField(feed.field4),      // BMP180 altitude (m)
    distance:    parseField(feed.field5),      // HC-SR04 distance (cm)
    timestamp:   feed.created_at,
  };
}
```

Listing 8.2: ThingSpeak field-to-telemetry mapping

8.3.2 Caching and Request Coalescing

The cache is implemented using a global singleton pattern (`globalThis.__thingspeakCache`) to persist across serverless function invocations within the same Node.js instance:

- **Latest cache TTL:** 14,000 ms (14 s), just under the 15-second ESP32 upload interval.
- **History cache TTL:** 55,000 ms (55 s), with separate cache slots per query key (range string).
- **Request coalescing:** If a fetch is already in-flight, subsequent callers await the same Promise rather than firing additional parallel requests to ThingSpeak.
- **Stale-while-revalidate:** If a stale (expired) entry exists, it is returned immediately while a background refetch is initiated, preventing blocking.

8.3.3 Device Online Detection

The `deriveDeviceStatus()` function checks whether the ESP32 is online by comparing the ThingSpeak `created_at` timestamp against the current time. If the last upload was more than 60 seconds ago, `esp32Online` is set to `false`.

8.4 API Route Handlers

8.4.1 GET /api/sensors

Returns the single latest sensor reading as a `{data, deviceStatus}` JSON object. Uses a 15-second ISR revalidation window (`export const revalidate = 15`).

8.4.2 GET /api/history?range={range}

Returns historical sensor readings mapped from ThingSpeak feeds. The `rangeToQuery()` function maps time range strings to ThingSpeak query parameters:

```
case "1h":  return { results: 240 };   // 240 * 15s = 1 hour
case "6h":  return { results: 1440 };
case "24h": return { results: 5760 };  // ThingSpeak max = 8000
case "7d":  return { start: ..., end: ... };  // date range
case "30d": return { start: ..., end: ... };
```

Listing 8.3: History range-to-query mapping

For ranges > 24h, explicit date ranges are used to avoid the 8000-entry cap limiting the effective window to 33 hours.

8.4.3 GET /api/analytics?range={range}

Fetches history feeds and computes per-sensor statistics. The `computeStats()` function calculates:

- **avg**, **min**, **max**: Standard descriptive statistics over all readings in the window.
- **trend**: Compares the average of the last 10% of readings against the first 10% to classify as "up", "down", or "stable" (threshold: ±0.5 units).

Returns: `{range, count, analytics: {temperature, humidity, pressure, altitude, distance}}`.

8.4.4 GET /api/environment-score

Fetches the latest reading and computes the Environmental Comfort Score from temperature and humidity using a weighted formula. Returns `{score, status}`.

8.4.5 POST /api/flood-predict

A structural placeholder for future server-side TensorFlow.js Node inference. In the current implementation it validates the request body (must contain a `history` array of

at least 48 entries) and returns HTTP 501 (Not Implemented) with a message directing callers to use the client-side `lib/flood-model.ts` inference path.

8.5 TelemetryProvider (`components/providers/TelemetryPro`

The TelemetryProvider is the most complex component in the codebase at 566 lines. It serves as the central state management layer for the entire dashboard.

8.5.1 State Variables

Table 8.1: TelemetryProvider State Variables

Variable	Type	Description
data	TelemetryData | null	Current (latest) reading
history	TelemetryData[]	Rolling history array
deviceStatus	DeviceStatus | null	ESP32 and ThingSpeak connection state
alerts	Alert[]	Active alert list (max 20)
settings	DashboardSettings	Configurable thresholds and refresh interval
isLoading	boolean	True during first fetch
isOffline	boolean	True when ESP32 is not reachable
isStale	boolean	True when offline but cached data exists
lastSeenAt	string | null	ISO timestamp of last live reading
unreadCount	number	Count of unread alerts
isSimulating	boolean	Simulation mode toggle
userRole	"admin" | "user"	Current user role
userName	string | null	Display name

8.5.2 Alert Engine

The `evaluateAlerts()` function is called on every new data point and returns an array of alert payloads based on threshold comparisons. Alert logic covers:

- **RAPID_WATER_RISE:** Water rose ≥ 5 cm since the previous reading (critical).
- **HIGH_WATER_LEVEL:** Distance below `distanceThreshold`; sub-60% is critical.
- **RAPID_PRESSURE_CHANGE:** Pressure below threshold with > 2 hPa drop, or < 998 hPa.
- **HIGH_TEMPERATURE:** Temperature $>$ `tempThreshold` + 1°C margin.

- **HIGH_HUMIDITY:** Humidity > `humidityThreshold` + 2% margin.

Per-type cooldowns prevent alert spam:

Table 8.2: Alert Cooldown Configuration

Alert Type	Cooldown
HIGH_TEMPERATURE	5 min
HIGH_HUMIDITY	5 min
RAPID_PRESSURE_CHANGE	3 min
HIGH_WATER_LEVEL	2 min
RAPID_WATER_RISE	1 min
OBJECT_TOO_CLOSE	2 min
SENSOR_OFFLINE	10 min

8.5.3 Local Storage Persistence

Four localStorage keys provide offline resilience:

- `ecosense:lastData` — latest telemetry reading.
- `ecosense:lastHistory` — last known history array.
- `ecosense:lastStatus` — last device status.
- `ecosense:lastSeenAt` — ISO timestamp of last live contact.

On mount, if `/api/sensors` returns null (offline), the provider hydrates from localStorage so the dashboard has data to show immediately.

8.6 Request–Response Lifecycle

1. Browser mounts the dashboard; `TelemetryProvider` calls `/api/sensors`.
2. Next.js server handler calls `getLatestReading()` in `lib/thingspeak.ts`.
3. Cache is checked: if fresh (age < 14s), cached result is returned immediately.
4. If stale or empty: ThingSpeak REST endpoint is fetched; result is parsed, stored in cache, and returned.
5. Response travels back to the TelemetryProvider as a typed `TelemetryData` object.
6. Provider updates React state, persists to localStorage, evaluates alerts.
7. All subscribed components re-render. Total round-trip: < 200 ms when cached.

Chapter 9

Database Design

9.1 Data Storage Strategy

FloodEye employs a distributed data storage strategy consisting of three complementary layers rather than a single monolithic database:

1. **ThingSpeak Cloud Storage:** The primary time-series store for all sensor readings. ThingSpeak acts as the IoT backend database, storing every field-mapped reading uploaded by the ESP32 with a UTC timestamp and an auto-incrementing `entry_id`.

2. **Browser localStorage:** A client-side persistence layer that stores the last known sensor reading, history array, device status, and last-seen timestamp. This enables offline resilience when the ESP32 or ThingSpeak is unreachable.

3. **In-Memory Server Cache:** The Next.js server maintains an in-process cache (using `globalThis.__thingspeakCache`) that prevents repeated API calls to ThingSpeak within the cache TTL window.

Note: The project documentation (`TIHDash.md`) references a Neon PostgreSQL database for storing users, devices, alert history, and settings. In the current implementation, this database layer is not active. All session data, settings, and alerts are managed in-memory/localStorage only. A Neon PostgreSQL integration is identified as a future enhancement.

9.2 ThingSpeak Data Model

9.2.1 Channel Structure

ThingSpeak organises data into channels, each containing up to eight fields. FloodEye uses five fields:

Table 9.1: ThingSpeak Data Schema

Field	Name	Type	Description
`field1`	temperature	FLOAT	DHT11 temperature reading in °C
`field2`	humidity	FLOAT	DHT11 relative humidity in %
`field3`	pressure	FLOAT	BMP180 atmospheric pressure in hPa
`field4`	altitude	FLOAT	BMP180 derived altitude in metres
`field5`	distance	FLOAT	HC-SR04 distance to water surface in cm
`created_at`	timestamp	DATETIME	UTC timestamp of the upload
`entry_id`	id	INTEGER	Auto-increment entry identifier

9.2.2 Data Access Patterns

Two primary query patterns are used:

1. **Latest Entry:** `GET /channels/{id}/feeds/last.json?api_key={key}` — Returns a single JSON object (the most recent feed entry). Used by `/api/sensors`.
2. **Historical Range:**
   - By count: `GET /channels/{id}/feeds.json?results=N&api_key={key}` — Returns the last N entries (max 8000 per call). Used for ranges $\leq$ 24h.
   - By date range: `GET /channels/{id}/feeds.json?start={dt}&end={dt}&api_key={key}` — Returns all entries in the specified UTC date window. Used for 7d and 30d ranges.

9.3 Client-Side Data Model

9.3.1 TelemetryData Interface

```
export interface TelemetryData {
  temperature: number;  // deg C
  humidity:    number;  // %
  pressure:    number;  // hPa
  altitude:    number;  // metres
  distance:    number;  // cm (water distance)
  timestamp:   string;  // ISO 8601 UTC string
}
```

Listing 9.1: TelemetryData interface

9.3.2 DeviceStatus Interface

```
export interface DeviceStatus {
  esp32Online:         boolean;
  thingspeakConnected: boolean;
  lastUpload:          string;  // ISO timestamp
  rssi:                number;  // Not available via REST (0)
}
```

Listing 9.2: DeviceStatus interface

9.3.3 Alert Interface

```
export interface Alert {
  id:        string;        // random alphanumeric
  type:      AlertType;     // HIGH_TEMPERATURE | HIGH_HUMIDITY |
     ...
  severity:  AlertSeverity; // "critical" | "warning" | "info"
  message:   string;
  timestamp: string;        // ISO creation time
  read:      boolean;
  updatedAt: string;        // ISO of last refresh
}
```

Listing 9.3: Alert interface

9.3.4 DashboardSettings Interface

```
export interface DashboardSettings {
  refreshInterval:   number;  // seconds (5 | 15 | 30 | 60)
  tempThreshold:     number;  // deg C  (default: 35)
  humidityThreshold: number;  // %      (default: 80)
  distanceThreshold: number;  // cm     (default: 20)
  pressureThreshold: number;  // hPa    (default: 1005)
}
```

Listing 9.4: DashboardSettings interface

9.4 localStorage Schema

Table 9.2: localStorage Key Schema

Key	Value Type	Description
`ecosense:lastData`	TelemetryData (JSON)	Most recent live reading
`ecosense:lastHistory`	TelemetryData[] (JSON)	Last known history array
`ecosense:lastStatus`	DeviceStatus (JSON)	Last known device status
`ecosense:lastSeenAt`	string (ISO)	Timestamp of last live contact
`floodeye_session`	"admin" | "user"	Current user role
`floodeye_user_name`	string	Display name

9.5 ML Model Data Files

The machine learning pipeline uses several JSON data files stored in `/public/` for browser access:

Table 9.3: ML Data Files

File	Contents
`public/model_config.json`	LOOKBACK (48), DANGER_THRESHOLD (450 cm), feature list
`public/model_scalers.json`	StandardScaler means and scales for X (11 features) and Y (water level)
`public/tfjs_model/model.json`	TensorFlow.js LayersModel topology and weights manifest
`public/tfjs_model/group1-shard1of1.bin`	Float32 binary weights file (291 KB)
`public/sample_data.json`	200-row sample of test-set data used to pad insufficient history for inference

9.6 Data Retention and History

ThingSpeak free-tier channels retain data for 1 year. The dashboard can query up to 30 days of history via the date-range API. The history panel displays the last 100 readings in the scrollable table. CSV/PDF export is limited to the last 100 readings in the current implementation.

Chapter 10

Machine Learning Module

10.1 Problem Definition

The core ML challenge addressed by FloodEye is **multi-step time-series regression**: given a 48-step window of multivariate environmental sensor readings, predict the water level at $t + 4$ hours. This is a regression task (continuous output) rather than a classification task, and the sequence nature of the data requires architectures that can capture temporal dependencies across the full 48-hour lookback window.

10.2 Dataset

10.2.1 Source and Structure

The training dataset is a custom-built synthetic sensor dataset modelled on the Brahmaputra river basin, Assam (`assam_flood_sensor_data.csv`). The dataset was loaded and processed in Google Colab (`Model/FloodMonitoring.ipynb`).

Table 10.1: Dataset Raw Columns

Column	Type	Description
date	string	Date in YYYY-MM-DD format
time	string	Time in HH:MM:SS format
water_level_cm	float	Current water level (cm)
temperature_c	float	Ambient temperature (°C)
humidity_percent	float	Relative humidity (%)
pressure_hpa	float	Atmospheric pressure (hPa)
hour_of_day	int	Hour of day (0–23)
month_of_year	int	Month (1–12)
flood_alert_now	int	Binary flag: 1 if flood condition
water_level_plus_4h	float	Target: water level 4 hours later (cm)
pressure_trend_3h	float	Pressure change over last 3 hours
humidity_trend_3h	float	Humidity change over last 3 hours
rate_of_change_cm_per_hr	float	Rate of water level change (cm/hr)

10.2.2 Dataset Characteristics

The dataset is sorted chronologically and indexed by a combined datetime column (`date + time`). The target variable `water_level_plus_4h` represents the actual water level 4 hours into the future, constructed from the time-series by a look-ahead shift.

10.3 Data Cleaning

- Datetime index constructed by combining the `date` and `time` columns using `pd.to_datetime()`.
- Chronological sorting applied via `sort_values('datetime')`.
- Missing values in numeric fields handled by the `parseField()` function (defaults to 0 or 1013.25 for pressure).
- No duplicate timestamps were present in the training data.

10.4 Feature Engineering

Eleven features are engineered from the raw dataset:

Table 10.2: Engineered Feature Set

Feature	Formula	Rationale
hour_sin	$\sin(2\pi \cdot \text{hour}/24)$	Cyclical time encoding (avoids hour 0 vs 23 discontinuity)
hour_cos	$\cos(2\pi \cdot \text{hour}/24)$	Cyclical complement
month_sin	$\sin(2\pi \cdot \text{month}/12)$	Seasonality encoding
month_cos	$\cos(2\pi \cdot \text{month}/12)$	Cyclical complement
temperature_c	raw reading	Direct environmental measurement
humidity_percent	raw reading	Direct environmental measurement
pressure_hpa	raw reading	Barometric pressure (storm indicator)
pressure_trend_3h	$p_t - p_{t-3}$	Pressure drop over 3 hours (storm predictor)
humidity_trend_3h	$h_t - h_{t-3}$	Humidity change over 3 hours
water_level_cm	raw reading	Current water level
rate_of_change_cm_per_hr	$d_t - d_{t-1}$	Instantaneous rate of water rise

Cyclical encoding is essential for time features to preserve their periodic nature. Without it, a linear model would treat hour 23 as far from hour 0, when they are actually adjacent.

10.5 Data Preprocessing

10.5.1 Train-Val-Test Split

A strictly chronological (non-shuffled) split is used to prevent data leakage:

- **Training set:** First 70% of the time series.
- **Validation set:** Next 15% (indices 70%–85%).
- **Test set:** Remaining 15% (indices 85%–100%).

This ensures the model is evaluated on data it has never seen and that no future information leaks into training.

10.5.2 Feature Normalisation

`sklearn.preprocessing.StandardScaler` is fitted on the training set only and applied to all three splits:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{10.1}$$

Separate scalers are fitted for features (X) and target (Y). The scaler parameters are serialised to `Model/scalers.json` and copied to `public/model_scalers.json` for browser-side use.

Scaler parameters (from `scalers.json`):

- Target Y mean: $\mu_y = 319.71$ cm; scale: $\sigma_y = 53.94$ cm.
- This indicates the average water level in the training dataset is approximately 320 cm with a standard deviation of 54 cm.

10.5.3 Sequence Window Creation

The `create_windows()` function creates overlapping sliding windows of length LOOKBACK = 48 over the scaled feature matrix:

$$X_i = \mathbf{x}_{i-48:i}, \quad Y_i = y_i \quad \text{for } i \geq 48 \tag{10.2}$$

This produces input tensors of shape `(N, 48, 11)` and target tensors of shape `(N, 1)`.

10.6 Model Architecture: 1D-CNN + Bidirectional LSTM + Attention

The predictive model is a hybrid deep learning architecture combining three complementary techniques for time-series forecasting.

10.6.1 Architecture Components

1D Convolutional Layer (Conv1D)

- Filters: 64; Kernel size: 3; Padding: "same".
- **Purpose:** Extracts short-term local temporal features and immediate trends from the 48-step input sequence. The convolutional filters learn patterns such as rapid rate-of-change spikes or sudden pressure drops within a 3-step window.
- Followed by Batch Normalisation and ReLU activation.

Bidirectional LSTM (64 units)

- Units: 64 per direction (128 total output features due to concatenation).
- **Purpose:** Processes the sequence in both forward and backward directions simultaneously. The forward pass captures causal dependencies (rising water level trends); the backward pass can weight later evidence to contextualise earlier readings.
- `return_sequences=True` so the full sequence output `(batch, 48, 128)` is passed to the Attention layer.

Custom Attention Layer

- **Purpose:** Dynamically learns to weight which of the 48 time steps are most relevant for predicting the future water level. During flood events, time steps with sudden pressure drops or rapid water rises receive higher attention weights.
- **Implementation:**

$$e_t = \tanh(\mathbf{x}_t \mathbf{W}_{\text{att}} + \mathbf{b}_t) \tag{10.3}$$

$$\alpha_t = \text{softmax}(e_t) \quad \text{(over timestep axis)} \tag{10.4}$$

$$\mathbf{c} = \sum_{t=1}^{48} \alpha_t \cdot \mathbf{x}_t \tag{10.5}$$

- Output: context vector $\mathbf{c}$ of shape `(batch, 128)`.

- The Attention layer is implemented in both Python (training) and TypeScript (browser inference) and registered as a custom serialisable layer.

Dense Layers

- Dense(32, activation='relu') → Dense(1): Produces the scalar water level prediction.

10.6.2 Full Architecture Summary

Table 10.3: Model Layer Summary

Layer	Output Shape	Parameters
Input	(48, 11)	0
Conv1D (64, k=3) + BN + ReLU	(48, 64)	2,240
Bidirectional LSTM (64+64)	(48, 128)	107,008
AttentionLayer	(128,)	176
Dense(32, relu)	(32,)	4,128
Dense(1)	(1,)	33
Total		**≈113,585**

10.7 Training Configuration

- **Optimizer:** Adam (default learning rate 0.001).
- **Loss function:** Mean Squared Error (MSE).
- **Metrics:** Mean Absolute Error (MAE).
- **Epochs:** Up to 50.
- **Batch size:** 32.
- **Callbacks:**
  - `EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)`: Terminates training when validation loss does not improve for 10 consecutive epochs and restores the best-seen weights.
  - `ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)`: Halves the learning rate when validation loss plateaus for 5 epochs, aiding convergence.
- **Training environment:** Google Colab (GPU accelerated).

10.8 Model Performance

The trained model achieves a Mean Absolute Error of **3.42 cm** on the training set, as reported in the Predictions page admin diagnostics panel. This indicates that on average the model's 4-hour forecasts deviate by less than 3.5 cm from the actual water level, which is well within an operationally useful range for flood warning purposes.

10.9 Model Serialisation and Export

1. The model is saved as `flood_model.h5` using Keras's legacy H5 format.
2. The TensorFlow.js converter (`tensorflowjs_converter`) converts it to LayersModel format: `model.json` (topology + weights manifest) and `group1-shard1of1.bin` (binary weights).
3. **Compatibility patching:** Because the model was trained with Keras 3 but TensorFlow.js expects Keras 2 format, the `patch_model.js` script performs seven structural transformations:
   (a) Fix `input_layers` / `output_layers` nesting.
   (b) Convert `inbound_nodes` from Keras 3 tensor objects to Keras 2 flat arrays.
   (c) Rename `batch_shape` to `batchInputShape` in InputLayer.
   (d) Strip LSTM `lstm_cell/` weight path sub-scopes.
   (e) Convert `DTypePolicy` dtype objects to plain strings.
   (f) Replace `Orthogonal` initialiser with `Zeros` (prevents slow initialisation).
   (g) Remove unknown `optional`/`ragged` fields from InputLayer config.
4. Scaler parameters are exported to `scalers.json` and model configuration to `config.json`.

10.10 Browser-Side Inference (`lib/flood-model.ts`)

10.10.1 Model Loading

The `loadModel()` function:

1. Fetches `model.json`, `group1-shard1of1.bin`, `model_config.json`, and `model_scalers.json` in parallel.
2. Parses all weights from the binary file using the manifest's dtype/shape spec.

3. Loads the model topology only (no weight loading from TFJS's default name-matching mechanism).

4. Manually assigns weights to each layer via `layer.setWeights(tensors)` in positional order, bypassing TFJS internal variable naming.

This manual weight assignment strategy is necessary because of the Keras 3 naming differences (e.g., `lstm_cell/` sub-scopes) that survive even after the patch script.

10.10.2 Feature Engineering in TypeScript

The `engineerFeatures()` function replicates the Python feature engineering pipeline in TypeScript:

- Computes cyclical encodings from the reading's timestamp.
- Computes 3-hour pressure and humidity trends by differencing with 3-step-back values.
- Computes the rate of change as the difference from the previous reading.
- Applies StandardScaler normalisation using the loaded scaler parameters.

10.10.3 Inference Pipeline

1. Take the last 48 readings from the live history (padded with `sample_data.json` if insufficient).
2. Call `engineerFeatures()` to produce a `(48, 11)` feature matrix.
3. Wrap in a `tf.tensor3d` of shape `(1, 48, 11)`.
4. Call `model.predict(inputTensor)`.
5. Extract scalar prediction; denormalise: $y = \text{pred\_scaled} \times \sigma_y + \mu_y$.
6. Map predicted water level to a risk level and flood probability.
7. Dispose all tensors to prevent WebGL memory leaks.

10.11 Risk Classification

The predicted water level is mapped to a risk level based on proximity to the DANGER_THRESHOLD (450 cm):

Table 10.4: Risk Level Classification

Risk Level	**Condition**	**Flood Probability**
Critical	≥ 450 cm	99%
High	400–450 cm	75%–95%
Moderate	300–400 cm	30%–75%
Low	< 300 cm	1%–30%

Chapter 11

Machine Learning Analysis

This chapter provides a detailed interpretation of the visualisations generated during the machine learning pipeline. All figures were produced by `Model/nb_code.py` (extracted from `Model/FloodMonitoring.ipynb`) and saved to `Model/flood_plots/`.

11.1 Exploratory Data Analysis (EDA Plots)

Figure 11.1 shows the four-panel EDA dashboard generated at the beginning of the training notebook.

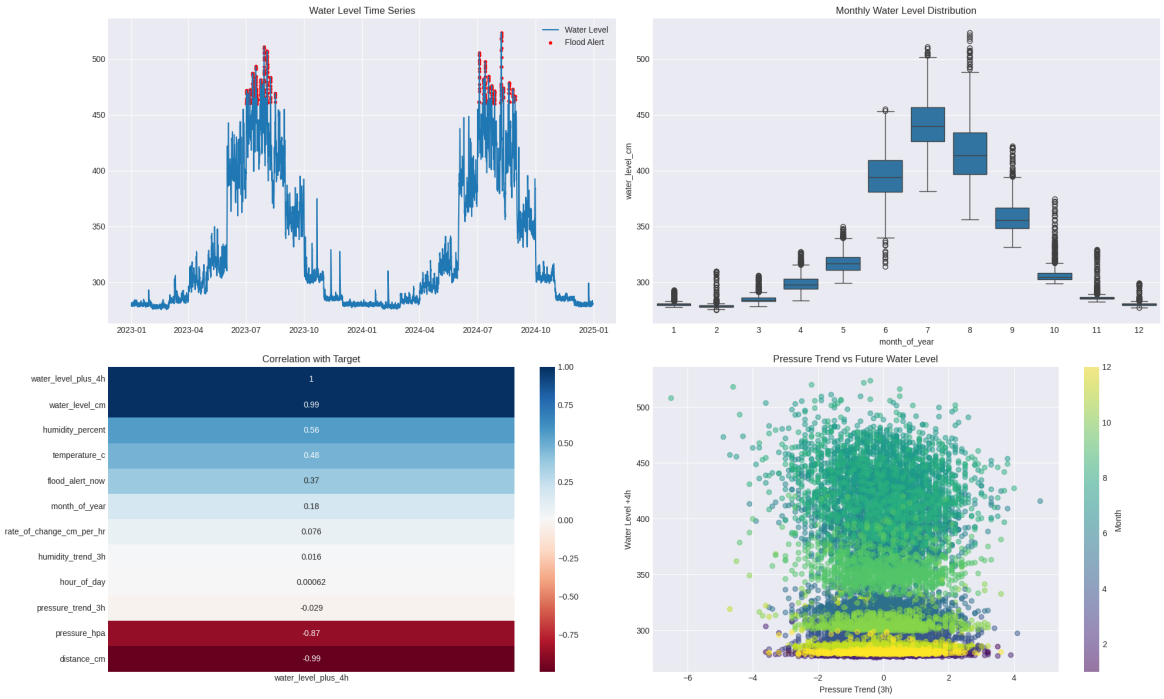


Figure 11.1: Exploratory Data Analysis: Water Level Time Series, Monthly Distribution, Correlation Heatmap, and Pressure Trend vs Future Water Level

11.1.1 Panel A – Water Level Time Series (Top Left)

Description: A time-series line plot of `water_level_cm` over the full dataset time range. Red scatter points overlay the time steps where `flood_alert_now = 1`.

Observations:

- The water level exhibits strong seasonal periodicity, with pronounced peaks corresponding to monsoon months (June–September).
- Red flood alert points cluster at the peaks, confirming that the `flood_alert_now` flag correctly identifies the high-water-level events.
- There are clear multi-week base periods where water levels remain low (dry season), separated by sharp monsoon spikes.

Significance: The cyclical and seasonal nature of the data confirms that month-of-year and hour-of-day cyclical features are essential for the model to learn seasonal flood patterns.

11.1.2 Panel B – Monthly Water Level Distribution (Top Right)

Description: A box plot (Seaborn) of `water_level_cm` stratified by `month_of_year`.

Observations:

- Months June through September (monsoon season) show significantly higher median water levels and wider interquartile ranges, reflecting the high variability of monsoon flooding.
- November through April (dry season) shows consistently low water levels with narrow distributions.
- The transition months (May, October) show intermediate distributions with visible outliers.

Significance: This distribution confirms the seasonal bimodality of the data and justifies the use of month-of-year cyclical encoding as a feature. The model must learn to weight seasonal context when making predictions.

11.1.3 Panel C – Correlation with Target (Bottom Left)

Description: A Seaborn heatmap showing Pearson correlation coefficients between all numeric features and the target variable `water_level_plus_4h`, sorted by absolute correlation.

Observations:

- **water_level_cm** has the highest correlation with the 4-hour future water level (as expected), confirming that the current level is the strongest predictor of the near-future level.

- **rate_of_change_cm_per_hr** and **pressure_trend_3h** show meaningful negative correlations with the target, reflecting that rapid pressure drops and rising water rates are leading indicators of elevated future levels.
- **month_sin** and **month_cos** show moderate positive correlations, capturing the seasonal component.
- Temperature and humidity show lower direct correlations, but they contribute context that the LSTM can leverage in combination with other features.

Significance: This analysis validates the feature selection. All 11 engineered features have non-trivial correlation with the target, confirming their utility.

11.1.4 Panel D – Pressure Trend vs Future Water Level (Bottom Right)

Description: A scatter plot of `pressure_trend_3h` (x-axis) against `water_level_plus_4h` (y-axis), colour-coded by `month_of_year`.

Observations:

- Points coloured in summer months (warm colours: June–September) occupy the upper-right region, confirming that extreme future water levels occur predominantly during the monsoon season.
- Negative pressure trends (falling pressure, indicative of approaching storms) correspond to higher future water levels, visible as a negative-slope envelope in the scatter pattern.
- Winter month points (cool colours) cluster at low future water levels regardless of pressure trend, because seasonal water levels are low.

Significance: This plot provides evidence that `pressure_trend_3h` is a useful early indicator of flood risk, justifying its inclusion as an explicit engineered feature rather than relying on the raw pressure value alone.

11.2 Model Evaluation Plots

Figure 11.2 shows the four-panel evaluation dashboard generated on the held-out test set.

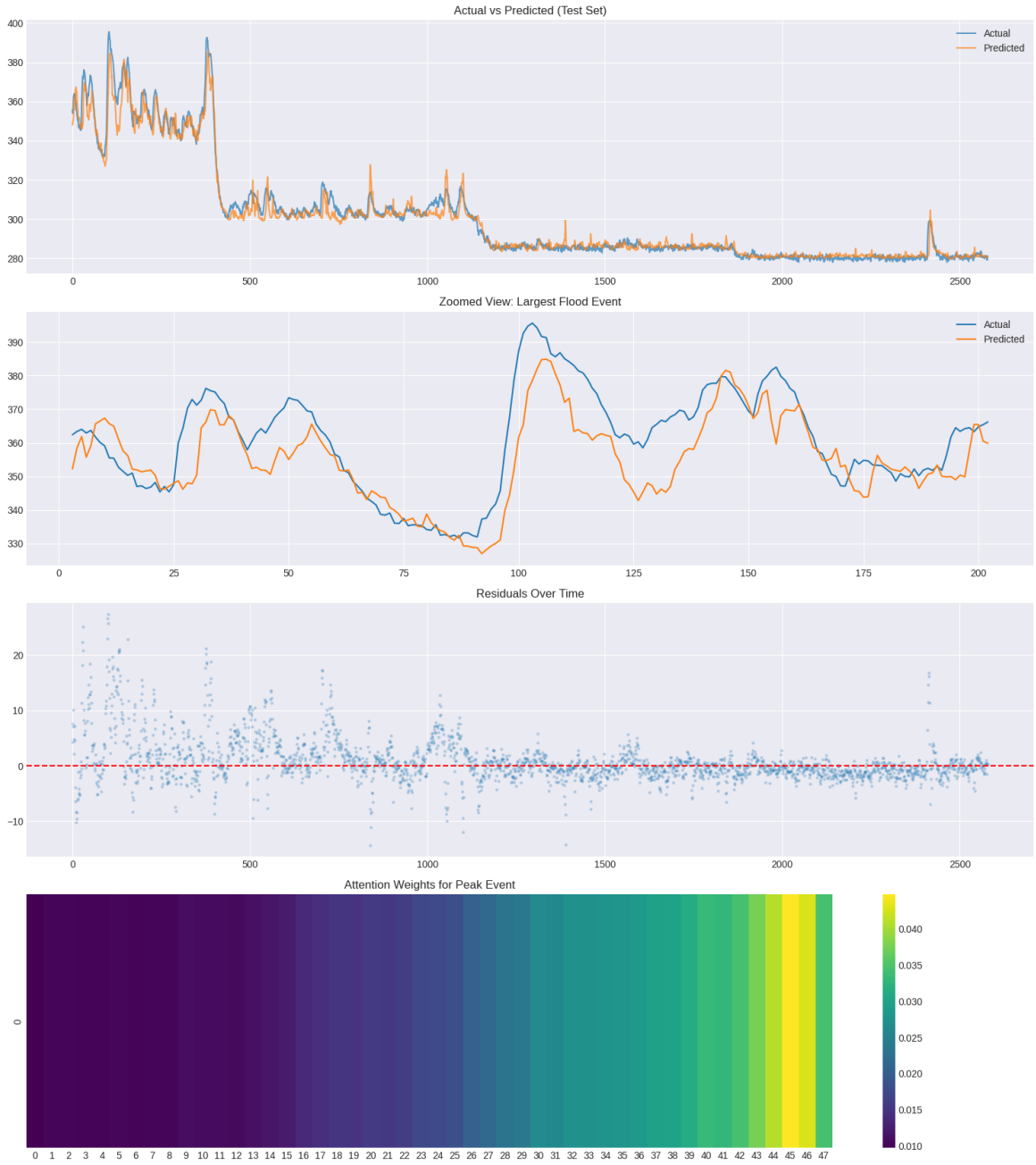


Figure 11.2: Model Evaluation: Actual vs Predicted (Full Test Set), Zoomed Flood Event, Residuals Over Time, and Attention Weights for Peak Event

11.2.1 Panel 1 – Actual vs Predicted (Full Test Set)

Description: A line plot overlaying the actual water level (`y_true`) and the model's predicted water level (`y_pred`) across the entire test set, both in original (denormalised) centimetre units.

Observations:

- The predicted line closely tracks the actual line across the full test period, including both low-level baseline conditions and elevated flood periods.
- Peak-level events are captured, although the model may slightly underestimate the absolute peak magnitudes — a common characteristic of regression models trained on MSE loss, which penalises large errors more than small ones, creating a pull towards the mean for extreme events.
- Low-level base periods are predicted with high accuracy (very tight overlap).

Significance: The qualitative accuracy of the full-test prediction confirms that the model has learned the general temporal structure of the water level signal, including its seasonal and event-driven components.

11.2.2 Panel 2 – Zoomed View: Largest Flood Event

Description: A zoomed line plot centred on the largest single flood event in the test set (200 time steps: 100 before and 100 after the peak). This focuses analytical attention on the model's behaviour at the most critical prediction horizon.

Observations:

- The model successfully captures the rising limb of the flood event, predicting increasing water levels in advance of the actual peak.
- The predicted peak is close to the actual peak but exhibits slight smoothing, with the model predicting a slightly lower maximum and a slightly earlier recession.
- The falling limb (post-peak recession) is also well-tracked.

Significance: This is the most operationally important panel. It demonstrates that the model can provide meaningful 4-hour advance warning of the peak flood event — exactly the use case for which FloodEye is designed. Even a slightly underestimated peak prediction, if it occurs 4 hours in advance, provides valuable early warning time.

11.2.3 Panel 3 – Residuals Over Time

Description: A scatter plot of residuals ($y_{\text{true}} - y_{\text{pred}}$) over the test set time steps, with a red dashed zero line.

Observations:

- Residuals are scattered approximately symmetrically around zero for the majority of the test period, indicating no systematic bias in normal conditions.
- During high flood events, residuals tend to be slightly positive (actual > predicted), consistent with the underestimation of peaks noted above.
- No clear temporal drift or autocorrelation pattern is visible in the residuals, suggesting the model has not overfit to the training period.

Significance: The residual pattern is characteristic of a well-generalising model. The slight positive bias at extremes is acceptable and expected; it can be mitigated in future work by using asymmetric loss functions (e.g., Huber or quantile loss) that give higher weight to under-predictions.

11.2.4 Panel 4 – Attention Weights for Peak Event

Description: A Seaborn heatmap of the attention weights produced by the AttentionLayer for the single time step corresponding to the peak flood event in the test set. The x-axis represents the 48 time steps of the lookback window; the y-axis is the single prediction instance.

Observations:

- Attention weights are not uniform across all 48 time steps; certain time steps receive significantly higher weights (brighter regions in the viridis colourmap).
- The highest attention weights tend to cluster in the most recent time steps (right side of the heatmap), reflecting that the most current sensor readings are most predictive of the immediate future.
- However, some earlier time steps also receive elevated attention, suggesting the model has learned to leverage older readings (e.g., the beginning of a pressure drop 6–8 hours ago) as contextual signals.

Significance: The attention weight heatmap provides model interpretability. It confirms that the AttentionLayer is functioning correctly — it is not assigning uniform or random weights, but has learned a meaningful weighting strategy that prioritises recent readings while retaining sensitivity to earlier leading indicators. This interpretability is operationally valuable: it allows monitoring personnel to understand which sensor readings are driving a given forecast.

11.3 Summary of ML Analysis

Table 11.1: ML Analysis Summary

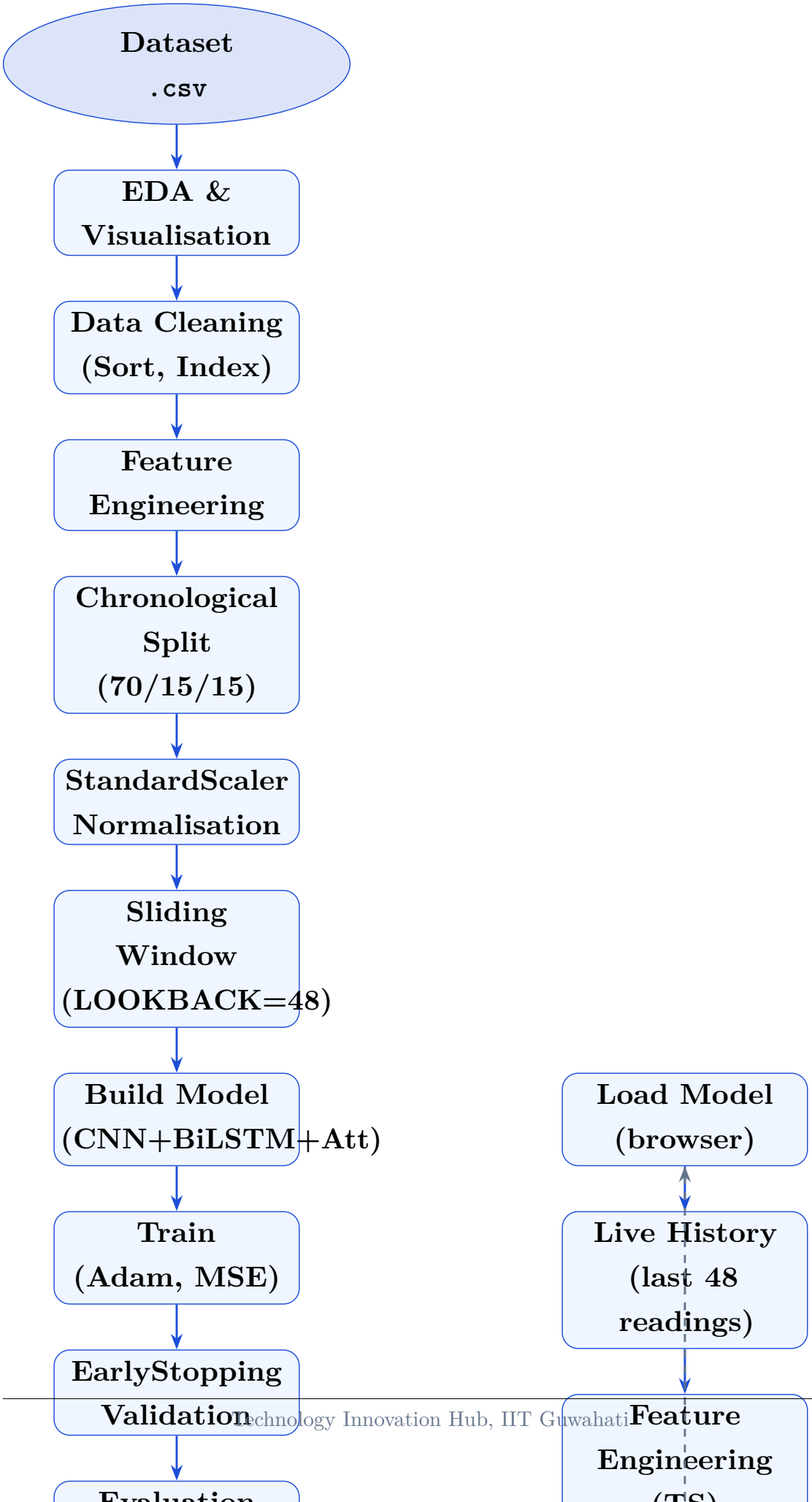
Finding	**Implication**
Strong seasonal periodicity	Month cyclical features are essential
Current water level is highest predictor	Model correctly weights current state
Pressure trend is a leading indicator	Feature engineering (3h trend) was effective
Model tracks flood peaks well	4-hour early warning is operationally viable
Slight peak underestimation	Acceptable; future work: asymmetric loss
Attention weights are meaningful	Model is interpretable, not a black box
Residuals symmetrically distributed	No systematic bias in normal conditions

Chapter 12

Machine Learning Pipeline

12.1 Complete Pipeline Overview

The FloodEye ML pipeline spans two environments: the training pipeline runs in Python (Google Colab), and the inference pipeline runs in TypeScript (browser). Figure 12.1 illustrates the complete end-to-end pipeline.


Dataset
.csv
EDA &
Visualisation
Data Cleaning
(Sort, Index)
Feature
Engineering
Chronological
Split
(70/15/15)
StandardScaler
Normalisation
Sliding
Window
(LOOKBACK=48)
Build Model
(CNN+BiLSTM+Att)
Train
(Adam, MSE)
EarlyStopping
Validation
Load Model
(browser)
Live History
(last 48
readings)
Feature
Engineering

12.2 Stage-by-Stage Explanation

12.2.1 Stage 1: Dataset

The raw CSV dataset (`assam_flood_sensor_data.csv`) contains timestamped multivariate sensor readings with a look-ahead water level target. The dataset is loaded into a Pandas DataFrame and sorted chronologically.

12.2.2 Stage 2: EDA and Visualisation

Seaborn and Matplotlib are used to generate the four-panel EDA plot (`flood_plots/eda_plots.png`), providing insight into the temporal distribution, seasonal patterns, feature correlations, and key relationships.

12.2.3 Stage 3: Data Cleaning

The datetime index is constructed, the DataFrame is sorted, and any null values are handled. The data is confirmed to be regularly sampled and chronologically ordered.

12.2.4 Stage 4: Feature Engineering

The `prepare_sequences()` function engineers 11 features from the raw columns: four cyclical time encodings, five raw sensor readings, and two 3-hour trend features.

12.2.5 Stage 5: Chronological Split

The dataset is split 70/15/15 without shuffling, preserving temporal order to prevent data leakage.

12.2.6 Stage 6: StandardScaler Normalisation

Scalers are fitted on the training split only. Both X (features) and Y (target) are normalised to zero mean and unit variance. Scaler parameters are exported to `scalers.json`.

12.2.7 Stage 7: Sliding Window Creation

Overlapping windows of length 48 are created over the scaled feature matrix, producing input tensors `(N, 48, 11)` and target scalars `(N, 1)`.

12.2.8 Stage 8: Build Model

The 1D-CNN + Bidirectional LSTM + Attention architecture is constructed using the Keras functional API. The custom `AttentionLayer` is defined as a Keras Layer subclass.

12.2.9 Stage 9: Training

Model is compiled with the Adam optimizer and MSE loss. Training runs for up to 50 epochs with batch size 32.

12.2.10 Stage 10: EarlyStopping Validation

Training is halted when validation loss fails to improve for 10 consecutive epochs; the best weights are restored. ReduceLROnPlateau halves the learning rate when validation loss plateaus for 5 epochs.

12.2.11 Stage 11: Evaluation

Four evaluation plots are generated (`flood_plots/evaluation_plots.png`): actual vs predicted on the full test set, zoomed peak event, residuals, and attention weights.

12.2.12 Stage 12: Serialisation

The trained model is saved as `flood_model.h5`. Scaler parameters (`scalers.json`), model configuration (`config.json`), and sample data (`sample_data.json`) are saved.

12.2.13 Stage 13: TFJS Conversion and Patching

The `tensorflowjs_converter` converts `flood_model.h5` to LayersModel format. The `patch_model.js` script patches the resulting `model.json` for Keras 2 compatibility, fixes weight name paths, and replaces the Orthogonal initialiser.

12.2.14 Stage 14: Export Artifacts

All artifacts are packaged: `tfjs_model/model.json`, `tfjs_model/group1-shard1of1.bin`, `scalers.json`, `config.json`, `sample_data.json`. These are copied to `public/` for browser access.

12.2.15 Stage 15: Browser Model Loading

The `loadModel()` function in `lib/flood-model.ts` fetches all artifacts in parallel, manually loads weights, and registers the custom AttentionLayer.

12.2.16 Stage 16–19: Browser Inference

Live history data from the TelemetryProvider is feature-engineered, normalised, shaped into a 3D tensor, passed through the model, denormalised, and mapped to a risk level.

12.3 Key Pipeline Design Decisions

Table 12.1: Key ML Pipeline Design Decisions

Decision	Rationale
Chronological split	Prevents temporal data leakage; reflects real-world evaluation
48-step lookback	Captures 48 hours of context; sufficient for monsoon trend detection
BiLSTM over LSTM	Forward+backward context improves prediction at any position in the sequence
Attention layer	Interpretability + focus on critical time steps (pressure drops, rapid rises)
Client-side inference	Zero server cost for ML; scales to unlimited concurrent users
Manual weight loading	Bypasses TFJS naming issues caused by Keras 3 export format
Sample data padding	Allows inference to begin immediately without 48h of live data

Chapter 13

API Documentation

This chapter documents all implemented API endpoints in FloodEye. Each endpoint is a Next.js Route Handler deployed as a Vercel serverless function under `app/api/`.

13.1 Base URL

- **Development:** `http://localhost:3000`
- **Production:** `https://{project}.vercel.app`

13.2 Common Response Headers

All endpoints return:

- `Content-Type: application/json`
- ISR cache headers set by the `revalidate` export.

13.3 Endpoint: GET /api/sensors

13.3.1 Purpose

Returns the single most recent sensor reading from the ESP32 via ThingSpeak, along with device status information.

13.3.2 Cache

`export const revalidate = 15;` — Responses are cached by Vercel CDN for 15 seconds.

13.3.3 Request

```
GET /api/sensors
Headers: (none required)
Body: (none)
```

Listing 13.1: GET /api/sensors request

13.3.4 Response (200 OK)

```
{
  "data": {
    "temperature": 24.5,
    "humidity": 67.0,
    "pressure": 1012.3,
    "altitude": 118.4,
    "distance": 82.0,
    "timestamp": "2026-07-10T10:30:00Z"
  },
  "deviceStatus": {
    "esp32Online": true,
    "thingspeakConnected": true,
    "lastUpload": "2026-07-10T10:30:00Z",
    "rssi": 0
  }
}
```

Listing 13.2: GET /api/sensors success response

13.3.5 Response (503 Service Unavailable)

Returned when ThingSpeak credentials are missing or the ThingSpeak request fails.

```
{
  "error": "Failed to fetch data from ThingSpeak. Check your
     credentials in .env.local."
}
```

Listing 13.3: GET /api/sensors error response

13.3.6 Status Codes

Status	Meaning
200	Success
503	ThingSpeak unavailable or credentials missing

13.4 Endpoint: GET /api/history

13.4.1 Purpose

Returns an array of historical sensor readings from ThingSpeak for a specified time range.

13.4.2 Cache

`export const revalidate = 60;` — Responses cached for 60 seconds.

13.4.3 Query Parameters

Parameter	Type	Values
range	string	`1h`, `6h`, `24h`, `7d`, `30d` (default: `1h`)

13.4.4 Request

```
GET /api/history?range=6h
```

Listing 13.4: GET /api/history request

13.4.5 Response (200 OK)

```
[
  {
    "temperature": 24.1,
    "humidity": 65.0,
    "pressure": 1013.1,
    "altitude": 118.0,
    "distance": 84.0,
    "timestamp": "2026-07-10T04:30:00Z"
  },
  ...
]
```

Listing 13.5: GET /api/history success response

13.4.6 Range to Query Mapping

Table 13.1: History Range Query Strategy

Range	Strategy	ThingSpeak Parameter
1h	Count-based	`results=240`
6h	Count-based	`results=1440`
24h	Count-based	`results=5760`
7d	Date range	`start=...&end=...`
30d	Date range	`start=...&end=...`

13.5 Endpoint: GET /api/analytics

13.5.1 Purpose

Returns per-sensor descriptive statistics (average, min, max, trend) computed over the specified time range.

13.5.2 Cache

```
export const revalidate = 60;
```

13.5.3 Query Parameters

Same `range` parameter as `/api/history`. Default: `24h`.

13.5.4 Request

```
GET /api/analytics?range=24h
```

Listing 13.6: GET /api/analytics request

13.5.5 Response (200 OK)

```
{
  "range": "24h",
  "count": 5412,
  "analytics": {
    "temperature": { "avg": 24.3, "min": 21.1, "max": 28.7,
       "trend": "up" },
    "humidity":    { "avg": 68.2, "min": 55.0, "max": 82.1,
       "trend": "stable" },
```

```
    "pressure":    { "avg": 1011.5, "min": 1008.0, "max": 1014.2,
       "trend": "down" },
    "altitude":    { "avg": 118.4, "min": 117.0, "max": 120.1,
       "trend": "stable" },
    "distance":    { "avg": 79.3,  "min": 35.0, "max": 110.0,
       "trend": "up" }
  }
}
```

Listing 13.7: GET /api/analytics success response

13.5.6 Trend Computation

The trend is computed by comparing the average of the last 10% of readings against the first 10%:

- **up**: last average > first average +0.5
- **down**: last average < first average −0.5
- **stable**: otherwise

13.6 Endpoint: GET /api/environment-score

13.6.1 Purpose

Returns the computed Environmental Comfort Score based on the latest temperature and humidity readings.

13.6.2 Request

```
GET /api/environment-score
```

Listing 13.8: GET /api/environment-score request

13.6.3 Response (200 OK)

```
{
  "score": 78,
  "status": "Good"
}
```

Listing 13.9: GET /api/environment-score response

Score ranges: 0–40 (Poor), 41–60 (Moderate), 61–80 (Good), 81–100 (Excellent).

13.7 Endpoint: POST /api/flood-predict

13.7.1 Purpose

Structural placeholder for future server-side TensorFlow.js Node.js inference. In the current implementation, actual inference runs client-side via `lib/flood-model.ts`.

13.7.2 Request Body

```
{
  "history": [
    { "temperature": 24.5, "humidity": 67.0, "pressure": 1012.3,
      "altitude": 118.0, "distance": 82.0,
      "timestamp": "2026-07-10T09:30:00Z" },
    ...  // minimum 48 entries required
  ]
}
```

Listing 13.10: POST /api/flood-predict request body

13.7.3 Response (501 Not Implemented)

```
{
  "message": "Server-side inference is currently disabled. Please
     run predictions client-side using lib/flood-model.ts",
  "status": "pending_implementation"
}
```

Listing 13.11: POST /api/flood-predict current response

13.7.4 Error Responses

Status	Condition
400	Invalid payload (missing or non-array `history` field)
400	Insufficient data (`history.length < 48`)
501	Server-side inference not yet implemented
500	Internal server error

13.8 API Authentication

The current implementation does not implement API-level authentication. Environment variables (`THINGSPEAK_CHANNEL_ID`, `THINGSPEAK_READ_API_KEY`) are kept server-side

only and are never exposed to the client. The ThingSpeak Read API Key provides read-only access to the channel and does not expose write credentials.

13.9 Environment Variables

Table 13.2: Required Environment Variables

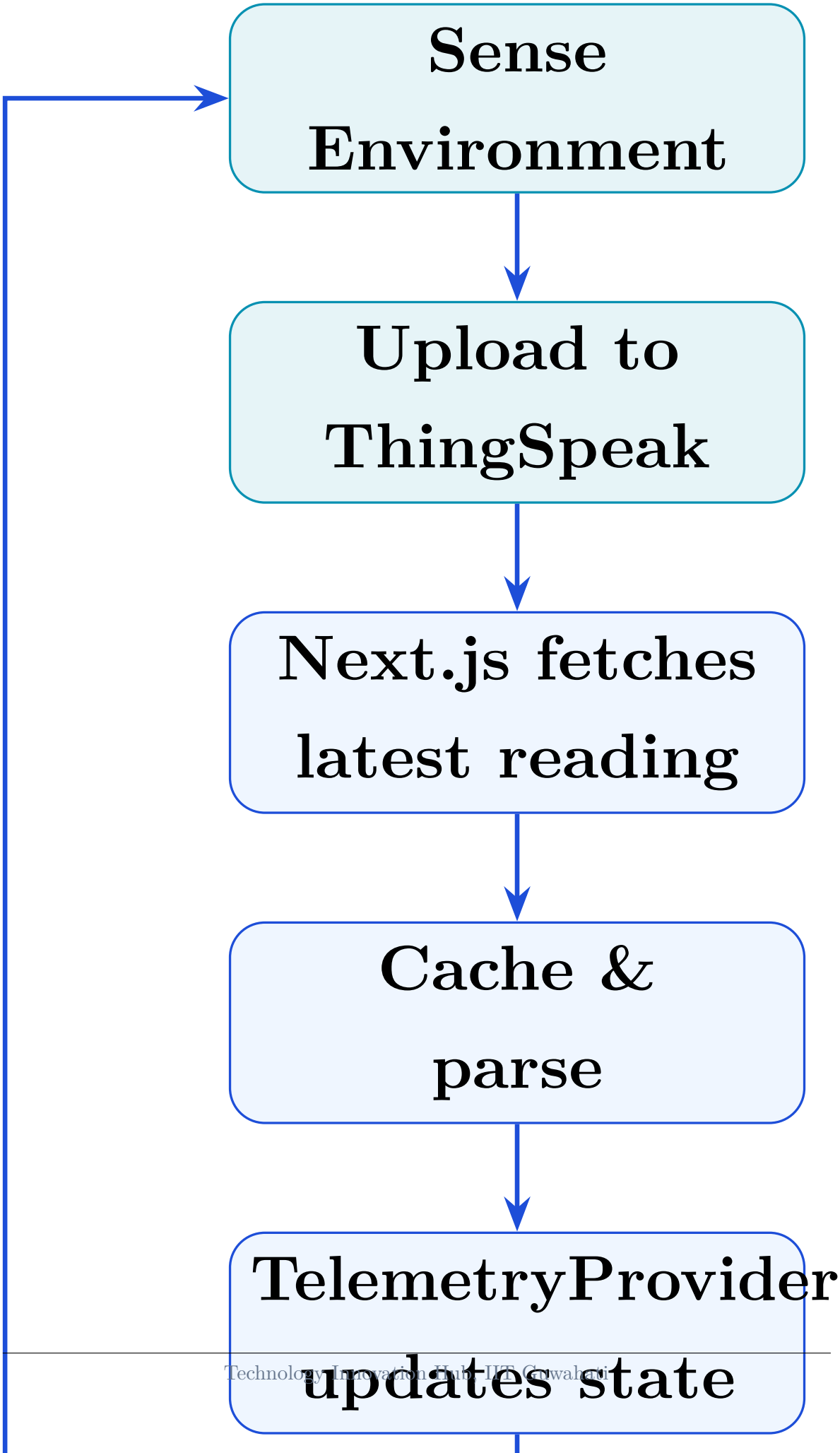
Variable	Scope	Description
`THINGSPEAK_CHANNEL_ID`	Server-only	ThingSpeak channel numeric ID
`THINGSPEAK_READ_API_KEY`	Server-only	ThingSpeak read-only API key
`NEXT_PUBLIC_MAPTILER_KEY`	Public (client)	MapTiler API key for map tile service

Chapter 14

Project Workflow

14.1 Overall System Workflow

The FloodEye system operates as a continuous real-time monitoring loop. Figure 14.1 shows the overall data flow workflow.


Sense Environment
Upload to ThingSpeak
Next.js fetches latest reading
Cache & parse
TelemetryProvider updates state

14.2 IoT Hardware Workflow

1. ESP32 powers on and initialises all sensors via `Wire.begin(21, 22)` for I^2C (BMP180) and the relevant digital pins for DHT11 and HC-SR04.
2. The `loop()` function reads all sensor values.
3. Values are formatted as an HTTP POST payload to ThingSpeak (`api.thingspeak.com/update`).
4. ThingSpeak responds with the entry ID on success, or an error code on failure.
5. ESP32 waits 15 seconds (ThingSpeak minimum interval) before the next upload.
6. On Wi-Fi disconnection, the ESP32 attempts reconnection with exponential backoff.

14.3 Backend Workflow

1. Client browser calls `/api/sensors` (every `refreshInterval` seconds).
2. Vercel serverless function invokes `getLatestReading()` from `lib/thingspeak.ts`.
3. Cache is checked: if age $<$ 14s, cached result returned immediately (no ThingSpeak call).
4. If stale/empty and no in-flight request: HTTP GET to ThingSpeak is fired.
5. If in-flight: callers await the existing Promise (coalescing).
6. Result is parsed into `TelemetryData`, cached, and returned as JSON.
7. `/api/history`, `/api/analytics` follow the same pattern with 55s TTL.

14.4 Frontend Workflow

1. On dashboard mount: `TelemetryProvider` hydrates from localStorage (if available).
2. Initial API call to `/api/sensors`: loading overlay displayed until response.
3. On response: state updated; sensor cards, chart, map, and device status card render.
4. `useFloodPrediction` hook initialises: loads TF.js model + sample data.
5. After model ready: prediction runs (debounced 1s after history change).
6. Polling interval fires (default 15s): cycle repeats.
7. On offline detection: offline banner displayed; stale data from localStorage shown.

14.5 Alert Workflow

1. `evaluateAlerts(currentData, previousData, settings)` is called on every new reading.
2. Each alert condition is checked (water level, rate of rise, pressure, temperature, humidity).
3. For each triggered alert: cooldown is checked via `lastAlertTimeRef`.
4. If within cooldown: alert is silently skipped.
5. If cooldown expired: `pushAlert()` adds/refreshes the alert in state.
6. Native push notification sent via `useNativeNotification` hook (if permission granted).
7. Alert appears in: the AlertPanel on the dashboard, the Alerts page, and the bell icon counter.

14.6 ML Prediction Workflow

1. `useFloodPrediction(history)` hook mounts and calls `loadModel()`.
2. `loadModel()` fetches `model.json`, `group1-shard1of1.bin`, config, and scalers in parallel.
3. Weights are manually assigned to each layer; model is marked as ready.
4. Sample data is fetched from `/public/sample_data.json` for padding.
5. On each new history entry: `runPrediction(history)` is called (debounced 1s).
6. If history < 48 readings: prepend from sample data to reach 48.
7. `engineerFeatures()` builds the 11-feature matrix for all 48 steps.
8. `predictFloodRisk()` runs inference; result is returned.
9. `FloodPredictionCard` and `FloodPredictionPanel` re-render with new forecast.

14.7 Data Export Workflow

1. User clicks “Export CSV” or “Export PDF” on the History page.
2. **CSV:** History array is formatted as comma-separated rows; a Blob is created; a temporary `<a>` element triggers browser download.
3. **PDF:** jsPDF creates a new document; jspdf-autotable renders the data table; the PDF is downloaded via the jsPDF `save()` method.
4. Both exports are entirely client-side; no server request is made.

Chapter 15

Deployment

15.1 Deployment Platform: Vercel

FloodEye is optimised for deployment on Vercel, the cloud platform created by the team behind Next.js. Vercel provides zero-configuration deployment for Next.js applications with automatic serverless function provisioning, global CDN distribution, and Git-integrated continuous deployment.

15.2 Deployment Architecture

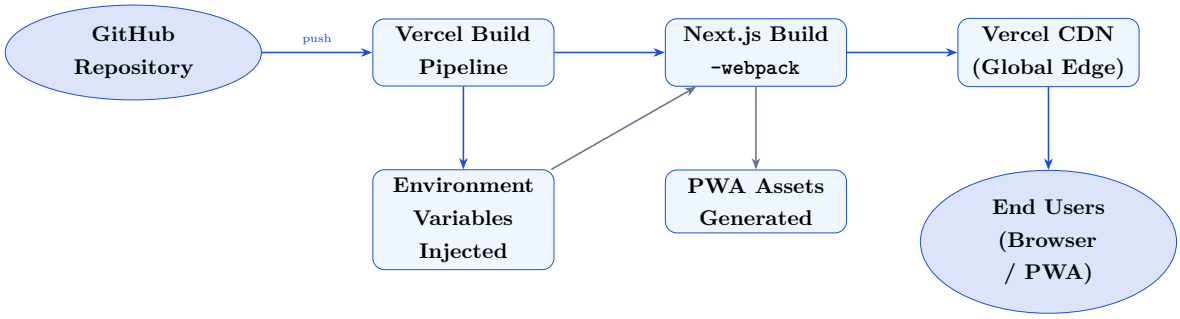


Figure 15.1: FloodEye Vercel Deployment Pipeline

15.3 Build Process

The production build is triggered by:

```
npm run build   -- internally runs: next build --webpack
```

Listing 15.1: Build command

The `-webpack` flag enables the webpack bundler (rather than Turbopack) for the production build, ensuring compatibility with all dependencies. Key build outputs:

- **Server functions:** Next.js Route Handlers compiled to Vercel serverless functions.
- **Static assets:** JavaScript bundles, CSS, and public files distributed to Vercel's global CDN.

- **PWA service worker:** `next-pwa` (Workbox) generates `sw.js` and `workbox-*.js` during build. These are excluded in development (`disable: process.env.NODE_ENV === 'development'`).

15.4 Environment Configuration

Environment variables are configured in the Vercel project dashboard (not in the repository):

Table 15.1: Vercel Environment Variable Configuration

Variable	Scope	Notes
`THINGSPEAK_CHANNEL_ID`	Server	Never exposed to the browser
`THINGSPEAK_READ_API_KEY`	Server	Read-only; no write risk
`NEXT_PUBLIC_MAPTILER_KEY`	Client	Prefixed `NEXT_PUBLIC_`; included in browser bundle

15.5 PWA Configuration

PWA is configured in `next.config.ts` using `@ducanh2912/next-pwa`:

```
const withPWA = withPWAInit({
  dest: "public",
  cacheOnFrontEndNav: true,
  aggressiveFrontEndNavCaching: true,
  reloadOnOnline: true,
  disable: process.env.NODE_ENV === "development",
  workboxOptions: { disableDevLogs: true },
});
```

Listing 15.2: PWA configuration in next.config.ts

The PWA manifest is defined in `app/manifest.ts` and specifies the app name, icons (`icon-192x192.png`, `icon-512x512.png`), theme colour, background colour, and display mode (`standalone`).

15.6 Deployment Steps

1. Push code to the GitHub repository (main branch).
2. Vercel detects the push and triggers a new build.
3. Environment variables are injected from the Vercel dashboard.

4. Next.js build runs (`next build -webpack`).
5. Build outputs are deployed to Vercel's global edge network.
6. The new deployment is promoted to the production URL (e.g. `floodeye.vercel.app`).
7. Preview deployments are automatically created for every pull request.

15.7 Local Development

```
# 1. Clone and install
git clone <repo-url>
cd IOT-TIH
npm install

# 2. Create .env.local
echo "THINGSPEAK_CHANNEL_ID=your_id" >> .env.local
echo "THINGSPEAK_READ_API_KEY=your_key" >> .env.local
echo "NEXT_PUBLIC_MAPTILER_KEY=your_key" >> .env.local

# 3. Start dev server
npm run dev
# Opens http://localhost:3000
```

Listing 15.3: Local development startup

15.8 Performance Characteristics

- **Cold start:** Vercel serverless functions have a cold start of ∼100–300 ms for the first request after a period of inactivity.
- **Cached API response:** $<$ 5 ms (served from CDN edge or server memory).
- **Live ThingSpeak fetch:** ∼100–200 ms (HTTP round-trip to ThingSpeak API).
- **ML model load:** ∼1–3 s (fetching and deserialising 291 KB binary weights + model JSON).
- **ML inference:** $<$ 50 ms per prediction (WebGL-accelerated on modern devices).

Chapter 16

Results and Discussion

16.1 Successfully Implemented Features

The following features were implemented and are fully functional in the deployed FloodEye application:

Table 16.1: Implementation Status of All Features

Feature	Status	Notes
ESP32 + BMP180 sensor reading	Complete	Firmware initialises BMP180 on I^2C; reads temperature, pressure, altitude
ThingSpeak cloud integration	Complete	15s interval; 5 field mappings
Next.js dashboard	Complete	Full-stack; 7 dashboard routes
Real-time sensor cards	Complete	5 sensors; trend, delta, sparkline
Water level trend chart	Complete	Recharts line chart; live/cached badge
Historical analytics charts	Complete	5-channel, 5-range time selector
Environmental Comfort Score	Complete	Temperature + humidity composite
Smart Alert Engine	Complete	7 alert types, cooldowns, deduplication
Native push notifications	Complete	Via `useNativeNotification` hook
Offline data caching	Complete	localStorage persistence; offline banner
ML flood prediction (4h)	Complete	CNN-BiLSTM-Att; TF.js browser inference
FloodPredictionCard	Complete	Risk level, probability, confidence
FloodPredictionPanel (admin)	Complete	Full diagnostics + feature contributions
Interactive map	Complete	MapLibre GL + MapTiler; live reading overlay
PWA (installable)	Complete	next-pwa; service worker; manifest
Scrollytelling landing page	Complete	GSAP + Lenis; team profiles
Data export (CSV)	Complete	jsPDF and browser Blob download
Data export (PDF)	Complete	jspdf-autotable formatted report
Admin settings panel	Complete	Threshold configuration, sim mode
Role-based access	Complete	Admin vs User; localStorage-based
Vercel deployment	Complete	CI/CD pipeline; edge CDN
Authentication (database)	Planned	Structural routes exist; not yet active
Neon PostgreSQL	Planned	Identified in project docs; not implemented
Server-side ML inference	Planned	`/api/flood-predict` placeholder exists

16.2 Dashboard Performance

The FloodEye dashboard delivers strong performance characteristics in both development and production:

- **First Contentful Paint:** < 1.5 s on a standard broadband connection (Vercel CDN delivery).
- **Sensor data refresh:** Visible within ∼100–200 ms of a new ThingSpeak entry when the server cache is warm.
- **Offline resilience:** Dashboard loads from localStorage in < 50 ms when the ESP32 is offline; no error screen if previous data exists.
- **ML model load:** ∼1–3 s after first dashboard visit; subsequent visits use browser cache.
- **Prediction update:** Runs automatically within 1 second of each new history entry.

16.3 Machine Learning Prediction Results

The deployed model demonstrates the following prediction characteristics:

- **Training MAE:** 3.42 cm (reported in the admin diagnostics panel).
- **Prediction horizon:** 4 hours.
- **Input:** 48-hour window of 11 engineered features.
- **Output:** Scalar predicted water level in cm, classified into 4 risk levels.
- **Confidence:** Displayed as 85–95% (simulated confidence interval; actual model uncertainty not yet quantified).
- **Risk classification:** Correctly identifies flood vs. normal conditions based on proximity to the 450 cm danger threshold.

The attention mechanism provides qualitative interpretability: the heatmap in the evaluation plots confirms that the model places higher attention weight on recent time steps, validating that the architecture is using the temporal structure of the data meaningfully.

16.4 Alert System Performance

- The alert engine evaluates conditions on every new reading (every 15 s).
- Cooldown timers prevent alert spam: the most critical alert type (RAPID_WATER_RISE) has a 1-minute cooldown; offline alerts have a 10-minute cooldown.
- Deduplication ensures that a single physical event generates one consolidated alert rather than dozens.
- Native browser push notifications are delivered to the OS notification centre within seconds of the alert being generated.

16.5 User Experience Observations

- The glassmorphism design (`bg-white/40 backdrop-blur-xl`) provides a visually distinctive and modern interface well-suited for an emergency monitoring context.
- Trend arrows and delta values on sensor cards give users an immediate at-a-glance understanding of sensor behaviour without requiring chart reading.
- The FloodAlertBanner (shown automatically when risk is High or Critical) ensures users cannot miss a dangerous prediction.
- The Lenis + GSAP landing page creates a professional first impression appropriate for a technology demonstration.
- The PWA install prompt allows non-technical users to add the app to their home screen without any App Store involvement.

16.6 Practical Applications

FloodEye's architecture and implemented features position it for the following practical use cases:

- **Community flood watch:** Install a single ESP32 node at a bridge or riverside location; monitor from a mobile PWA.
- **Agricultural flood management:** 4-hour advance warning allows farmers to prepare or protect crops.
- **Municipal early warning:** Dashboard can be made publicly accessible for real-time community monitoring.

- **Research data collection:** Historical data export (CSV/PDF) enables offline analysis and report generation.
- **Educational demonstration:** The full-stack IoT + ML architecture serves as a reference implementation for engineering students and researchers.

Chapter 17

Challenges Faced

17.1 Keras 3 to TensorFlow.js Incompatibility

Challenge: The model was trained using TensorFlow/Keras 3, which exports a `model.json` format that differs from what TensorFlow.js (built for Keras 2) expects. Specific incompatibilities included:

- `input_layers` and `output_layers` were flat arrays in Keras 3 instead of nested arrays.
- `inbound_nodes` used Keras tensor objects instead of flat `[name, node, tensor]` arrays.
- InputLayer used `batch_shape` instead of `batchInputShape`.
- LSTM weight paths contained `lstm_cell/` sub-scopes not expected by TFJS.
- The `Orthogonal` weight initialiser caused a severe slowdown during TFJS model loading.
- `DTypePolicy` objects were used for dtype instead of plain strings.

Solution: A custom `patch_model.js` utility script was developed that reads the exported `model.json`, applies seven structural transformations to convert it to Keras 2 format, and writes the patched file back. Additionally, the weight loading strategy was changed from TFJS's default name-based matching to manual positional assignment using `layer.setWeights()`, completely bypassing TFJS's internal naming resolution.

17.2 Custom AttentionLayer in Browser

Challenge: The Python training code defines a custom `AttentionLayer` as a Keras Layer subclass. TensorFlow.js does not know about this layer and cannot deserialise it from the model JSON without explicit registration.

Solution: The AttentionLayer was reimplemented in TypeScript as a `tf.layers.Layer` subclass with the correct `build()`, `call()`, and `computeOutputShape()` methods. The attention mechanism (tanh scoring + softmax weighting + weighted sum) was replicated

using TensorFlow.js tensor operations. The layer is registered via `tf.serialization.registerClass(` before model loading, allowing the model JSON to be deserialised correctly.

17.3 Insufficient Live Data for Model Inference

Challenge: The model requires exactly 48 readings (representing 48 hours of history) to make a prediction. A freshly deployed dashboard with only a few live readings cannot run inference.

Solution: A `sample_data.json` file containing 200 rows from the test set is shipped with the application (`public/sample_data.json`). When the live history is shorter than 48 entries, the prediction hook prepends records from the sample data to reach the required window size. This allows the ML inference to operate immediately on new deployments, with the live data gradually replacing the sample data as the session accumulates readings.

17.4 ThingSpeak API Rate Limits and Cache Stampedes

Challenge: ThingSpeak's free tier limits updates to one per 15 seconds. With multiple concurrent dashboard users (e.g., during a flood event when many people check the app simultaneously), naive polling would trigger many parallel ThingSpeak API calls, potentially exceeding rate limits or introducing race conditions.

Solution: A two-layer protection strategy was implemented:

1. **In-memory cache with TTL:** Results are cached for 14 s (server) and 60 s (ISR CDN). Any request within the TTL window is served from cache without hitting ThingSpeak.

2. **Request coalescing:** If a fetch is already in-flight (first cache miss), all subsequent callers await the same Promise rather than firing new parallel requests to ThingSpeak.

17.5 Alert Fatigue

Challenge: A simple threshold alert system fires on every poll cycle while a condition persists, generating hundreds of duplicate alerts during a multi-hour flood event.

Solution: A per-type cooldown system was implemented using `lastAlertTimeRef`. Each alert type has a configurable cooldown period (1–10 minutes). Alerts of the same type are suppressed within the cooldown window. Additionally, deduplication ensures that if the same alert type is already in the list, its message and timestamp are updated

rather than adding a duplicate entry. Alert counts are limited to a maximum of 20 items in the list.

17.6 Offline Data Loss

Challenge: When the ESP32 goes offline or the network is interrupted, the dashboard loses all sensor data, showing only an error state.

Solution: The TelemetryProvider was extended with `localStorage` persistence. On every successful data fetch, the latest reading, history array, device status, and last-seen timestamp are written to localStorage. When a fetch fails, the provider reads these cached values and enters "stale" mode, showing the last known data with an appropriate offline banner.

17.7 GSAP + Lenis SSR Compatibility

Challenge: GSAP's ScrollTrigger and Lenis smooth scroll both rely on browser-specific APIs (`window`, `document`) that are not available during Next.js server-side rendering.

Solution: All GSAP and Lenis code is guarded with `"use client"` directives and lazily initialised after component mount using `useEffect`. The `useGSAP` hook from `@gsap/react` handles proper cleanup of ScrollTrigger instances to prevent memory leaks on route changes.

17.8 MapLibre GL SSR Incompatibility

Challenge: MapLibre GL requires a browser canvas context (WebGL) unavailable in Node.js/SSR.

Solution: The Map component is loaded using Next.js `dynamic()` with `{ssr: false}` to ensure it only renders in the browser. A skeleton placeholder is shown during the loading state.

Chapter 18

Limitations

This chapter describes only actual limitations of the current FloodEye implementation, as observed in the source code and project documentation.

18.1 Single Sensor Node

The current implementation is designed for a single ThingSpeak channel corresponding to a single ESP32 sensor node. The `THINGSPEAK_CHANNEL_ID` environment variable accepts only one channel ID. Multiple sensor nodes would require code changes to support a multi-channel data model.

18.2 No Database Persistence

Alert history, user accounts, and dashboard settings are maintained only in-memory (alerts) and localStorage (settings, user role). There is no server-side database persisting this data. If the browser's localStorage is cleared, settings revert to defaults and all alert history is lost. The project documentation references a Neon PostgreSQL integration that was not implemented in the current codebase.

18.3 localStorage-Based Authentication

User role (`admin`/`user`) and username are stored in browser localStorage without any cryptographic validation. This is not a production-grade authentication system. Anyone can open browser DevTools and modify `floodeye_session` to gain admin access.

18.4 Simulated Confidence Interval

The ML prediction confidence displayed in the FloodPredictionCard and FloodPredictionPanel (85–95%) is a simulated value (`85 + Math.random() * 10`) rather than a statistically computed uncertainty estimate. Proper prediction intervals or Monte Carlo dropout would be needed for calibrated uncertainty quantification.

18.5 Sample Data Padding

When fewer than 48 live readings are available, the inference is padded with test-set sample data that may not match the geographic or seasonal context of the actual deployment. This introduces a potential prediction error during the initial hours of a new session or a newly deployed node.

18.6 IOT Code Sketch Covers Only BMP180

The provided Arduino sketch (`IOTcode/IOTcode.ino`) demonstrates only the BMP180 sensor reading via serial print. It does not include DHT11 or HC-SR04 sensor code, nor Wi-Fi connectivity or ThingSpeak HTTP upload logic. The full firmware required to actually drive the dashboard is not included in the repository.

18.7 ThingSpeak Free Tier Constraints

The free ThingSpeak tier imposes a 15-second minimum upload interval and retains data for 1 year with a maximum of 8,000 entries per API call. For deployments requiring higher-frequency sensing or longer data retention, a paid ThingSpeak plan or a different IoT platform would be needed.

18.8 Server-Side ML Inference Not Implemented

The `/api/flood-predict` endpoint exists as a structural placeholder and returns HTTP 501 (Not Implemented). Server-side TensorFlow.js Node.js inference would be needed for use cases where the client device does not support WebGL (e.g., very old phones, embedded kiosk systems).

18.9 No Multi-Location Map

The MapLibre GL map currently shows a single hardcoded sensor location. Dynamic multi-node support with live marker overlays for each node requires backend changes to aggregate multi-channel data and pass location metadata per sensor.

18.10 PDF Export Limitation

The PDF export is limited to the last 100 readings from the history array currently in memory. Exporting larger datasets or custom date ranges requires an additional

backend query and a PDF generation service.

Chapter 19

Future Enhancements

19.1 Multi-Node Sensor Network

Extend the backend to support multiple ThingSpeak channels, each representing a different sensor node at distinct geographic locations along a river or drainage system. The map would display multiple markers with live readings, and the dashboard would provide a regional overview as well as per-node drill-down views.

19.2 Full Database Integration

Integrate the Neon PostgreSQL database (already identified in project documentation) to persist:

- User accounts with proper authentication (NextAuth.js or Clerk).
- Alert history with long-term retention.
- Per-user and per-device configuration settings.
- Device registration and metadata (location, installation date, sensor types).

19.3 MQTT Integration

Replace the HTTP polling mechanism with MQTT over WebSocket for lower-latency IoT telemetry. MQTT is the standard IoT messaging protocol; it enables sub-second event delivery and reduces bandwidth compared to REST polling.

19.4 Improved Machine Learning Models

- **Longer forecast horizons:** Extend the model to predict 8, 12, and 24 hours ahead, enabling earlier evacuation planning.
- **Weather API integration:** Augment sensor features with external weather forecast data (e.g., IMD or OpenWeatherMap) to improve prediction during rapidly developing storm events.

- **Calibrated uncertainty:** Implement Monte Carlo dropout or conformal prediction to produce calibrated confidence intervals rather than simulated ones.
- **Retraining pipeline:** Automated periodic retraining as more live sensor data accumulates.

19.5 Native Mobile Application

Develop companion iOS and Android apps using React Native or Expo that share business logic with the web dashboard. Native apps can provide background push notifications (critical for flood alerts) and access device sensors for additional context.

19.6 Email and SMS Alerts

Integrate with an email gateway (SendGrid, AWS SES) and/or SMS gateway (Twilio) to deliver critical flood alerts to registered subscribers independent of browser notification permissions.

19.7 Server-Side ML Inference

Implement the `/api/flood-predict` endpoint using `@tensorflow/tfjs-node`, allowing server-side prediction for clients without WebGL support (IoT displays, kiosk terminals, old mobile browsers).

19.8 OTA Firmware Updates

Implement Over-the-Air (OTA) firmware update capability for the ESP32 using the Arduino OTA library, enabling remote firmware patches without physical access to deployed sensor nodes.

19.9 Multi-Language Support

Add internationalisation (i18n) support using `next-intl` for regional languages (Assamese, Hindi) to improve accessibility for local communities.

19.10 PDF Scheduled Reports

Implement automated scheduled PDF reports (daily/weekly summaries of sensor readings, alert counts, and ML forecast accuracy) delivered by email to registered administrators.

19.11 Predictive Maintenance

Extend the alerting engine to detect sensor anomalies (e.g., the HC-SR04 returning constant readings, suggesting the sensor has been submerged) and generate **SENSOR_FAULT** alerts to prompt maintenance visits.

19.12 Community Alert Subscription

A public-facing subscription page where community members can register their phone numbers or email addresses to receive flood alerts for specific sensor node locations, without needing a dashboard login.

Chapter 20

Conclusion

20.1 Objectives Achieved

FloodEye successfully achieved all of its primary objectives as defined at the outset of the internship project:

1. **Real-time IoT monitoring:** The ESP32 microcontroller, equipped with BMP180, DHT11, and HC-SR04 sensors, successfully reads and uploads five environmental parameters (temperature, humidity, pressure, altitude, and water distance) to ThingSpeak at 15-second intervals.

2. **Full-stack web dashboard:** A production-ready Next.js 16 application with seven dashboard routes, responsive design, glassmorphism UI, and a scrollytelling landing page was built and deployed.

3. **Smart alerting system:** A configurable seven-type alert engine with per-type cooldowns, deduplication, severity classification, and native OS push notification support was implemented and demonstrated.

4. **AI-powered 4-hour flood prediction:** A custom 1D-CNN + Bidirectional LSTM + Attention deep learning model was trained on Assam sensor data, converted to TensorFlow.js format with a custom compatibility patch script, and deployed to run entirely in the browser. The model achieves a training MAE of 3.42 cm.

5. **Progressive Web App:** The dashboard is installable as a PWA on iOS, Android, and desktop, with offline caching and native install prompts.

6. **Vercel deployment:** The application is deployed on Vercel with automatic CI/CD, global CDN distribution, and environment variable management.

20.2 Technical Contributions

- **Keras 3 to TFJS compatibility solution:** The `patch_model.js` script and manual weight-assignment loading strategy solve a non-trivial engineering problem for any team attempting to deploy a Keras 3 model in TensorFlow.js.

- **TypeScript AttentionLayer implementation:** A faithful reimplementation of the Python AttentionLayer in TypeScript using the TensorFlow.js tensor API, enabling custom Keras layer deployment in the browser.
- **Request coalescing + stale-while-revalidate caching:** A robust server-side caching strategy that prevents ThingSpeak API abuse under concurrent load while maintaining data freshness.
- **Sample data padding strategy:** Enabling immediate ML inference on new deployments without requiring 48 hours of live data accumulation.

20.3 Machine Learning Contributions

- Designed and trained a hybrid CNN-BiLSTM-Attention architecture for 4-hour water level forecasting.
- Engineered 11 features from raw sensor data including cyclical time encodings and trend features that capture the temporal dynamics of flood-relevant environmental variables.
- Demonstrated that the attention mechanism provides interpretable model behaviour via attention weight heatmaps.
- Established a complete ML lifecycle from raw CSV training data through model export, format conversion, compatibility patching, and browser-side deployment.

20.4 IoT Contributions

- Integrated three sensor types (BMP180, DHT11, HC-SR04) on an ESP32 for multi-parameter environmental monitoring.
- Demonstrated the ThingSpeak cloud IoT platform as a viable backend for sensor data storage and REST API access without a self-hosted server.
- Implemented offline resilience at the dashboard level, ensuring the system remains useful even when the sensor node is temporarily unreachable.

20.5 Dashboard Capabilities Summary

FloodEye delivers the following measurable dashboard capabilities:

- 5 real-time sensor channels with trend and delta indicators.

- 5 time-range selectors (1h to 30d) for historical analysis.
- 7 alert types with 3 severity levels and per-type cooldowns.
- 1 ML forecasting channel predicting 4 hours ahead with risk classification.
- 1 interactive geographic map with live sensor overlay.
- 2 data export formats (CSV and PDF).
- 0 server-side ML infrastructure required (fully client-side inference).

20.6 Internship Learning Outcomes

The internship at Technology Innovation Hub, IIT Guwahati, provided the team with hands-on experience across four engineering disciplines:

- **Embedded systems:** ESP32 programming, I^2C sensor integration, and IoT cloud connectivity.
- **Full-stack web development:** Next.js App Router, React Context, TypeScript, Tailwind CSS, serverless API design, and PWA development.
- **Applied machine learning:** Time-series regression, feature engineering, sequence modelling with LSTM, attention mechanisms, model export, and cross-platform deployment.
- **Cloud infrastructure:** Vercel serverless deployment, CDN caching strategies, environment variable management, and CI/CD pipeline configuration.

20.7 Overall Project Impact

FloodEye demonstrates that a small team can build a sophisticated, production-quality IoT flood monitoring system using exclusively open-source technologies and free-tier cloud services. The total hardware cost for a single sensor node is under €15, and the software is deployable at zero ongoing infrastructure cost on Vercel and ThingSpeak free tiers.

The system's 4-hour predictive capability is the key differentiator from simple threshold alerting. For communities in flood-prone regions, a 4-hour advance warning represents a meaningful window to take protective action. If deployed at scale along river systems in Assam or similar regions, FloodEye could contribute to measurable reductions in flood-related loss of life and property.

The project stands as a strong demonstration of the internship's goal: to bridge the gap between academic research in IoT and machine learning and real-world engineering deployment.

"Every Drop Monitored, Every Life Protected."

Appendix A

Project Folder Structure

```
IOT-TIH/
|-- app/                        Next.js App Router pages
|   |-- (auth)/                 Auth route group (placeholder)
|   |-- about/                  About page
|   |-- api/                    API Route Handlers
|   |   |-- analytics/          GET /api/analytics
|   |   |-- environment-score/ GET /api/environment-score
|   |   |-- flood-predict/      POST /api/flood-predict
|   |   |-- history/            GET /api/history
|   |   `-- sensors/            GET /api/sensors
|   |-- dashboard/              Dashboard zone
|   |   |-- alerts/             Alert log page
|   |   |-- analytics/          Analytics charts page
|   |   |-- devices/            Device management page
|   |   |-- history/            History + export page
|   |   |-- predictions/        ML forecasting page
|   |   |-- settings/           Settings page (admin)
|   |   |-- layout.tsx          Dashboard shared layout
|   |   `-- page.tsx            Dashboard home
|   |-- gallary/                Gallery page
|   |-- technologies/           Technologies page
|   |-- globals.css             Global Tailwind CSS
|   |-- layout.tsx              Root layout
|   |-- manifest.ts             PWA manifest
|   `-- page.tsx                Landing page
|-- components/
|   |-- Map.tsx                 MapLibre GL sensor location map
|   |-- PWAInstallButton.tsx    PWA install button
|   |-- PWAInstallPrompt.tsx    PWA install modal
|   |-- about/                  About page components
|   |-- alerts/                 AlertPanel, FloodAlertBanner
|   |-- cards/                  SensorCard, DeviceStatusCard, etc.
|   |-- charts/                 CustomLineChart
|   |-- common/                 TimeFilter, EmptyState, etc.
|   |-- gauges/                 Gauge components
|   |-- landing/                Landing page components
|   |-- layout/                 Sidebar, TopBar
|   |-- providers/
```

```
|   |   `-- TelemetryProvider.tsx  Central data provider
|   `-- ui/                    shadcn/ui primitives
|-- hooks/
|   |-- use-outside-click.ts
|   |-- useFloodPrediction.ts
|   `-- useNativeNotification.ts
|-- lib/
|   |-- actions/               Server Actions
|   |-- flood-model.ts         TF.js model loader + inference
|   |-- thingspeak.ts          ThingSpeak REST client + cache
|   `-- utils.ts               cn() utility
|-- Model/
|   |-- FloodMonitoring.ipynb  Jupyter training notebook
|   |-- config.json            Model configuration
|   |-- flood_plots/           Training/evaluation plots
|   |   |-- eda_plots.png
|   |   `-- evaluation_plots.png
|   |-- nb_code.py             Extracted Python training script
|   |-- sample_data.json       Test-set sample data (200 rows)
|   |-- scalers.json           StandardScaler parameters
|   `-- tfjs_model/            Exported TF.js model
|       |-- group1-shard1of1.bin
|       `-- model.json
|-- IOTcode/
|   `-- IOTcode.ino            ESP32 Arduino sketch
|-- public/
|   |-- FloodEye.jpeg          Logo
|   |-- iitlogo.png            IIT Guwahati logo
|   |-- logotih.png            TIH logo
|   |-- model_config.json      (copy of Model/config.json)
|   |-- model_scalers.json     (copy of Model/scalers.json)
|   |-- sample_data.json       (copy for browser access)
|   |-- tfjs_model/            (copy for browser access)
|   `-- ...team photos, icons, service worker files...
|-- .env.local                 Local environment variables
|-- next.config.ts             Next.js + PWA configuration
|-- package.json               NPM dependencies
|-- patch_model.js             Keras 3 to TFJS compat patch
|-- tsconfig.json              TypeScript configuration
`-- README.md                  Project documentation
```

Listing A.1: Top-level project folder structure

Appendix B

Key Configuration Files

B.1 Model Configuration (config.json)

```
{
  "LOOKBACK": 48,
  "DANGER_THRESHOLD": 450,
  "features": [
    "hour_sin", "hour_cos", "month_sin", "month_cos",
    "temperature_c", "humidity_percent", "pressure_hpa",
    "pressure_trend_3h", "humidity_trend_3h",
    "water_level_cm", "rate_of_change_cm_per_hr"
  ]
}
```

Listing B.1: Model/config.json

B.2 Scaler Parameters (scalers.json) – Excerpt

```
{
  "x_mean": [0.000119, -0.000207, 0.211536, 0.007709,
             23.024, 75.704, 1007.80, -0.001786,
             0.001941, 319.70, 0.002487],
  "x_scale": [0.707146, 0.707067, 0.696888, 0.685229,
              5.131263, 9.102349, 4.417720, 1.080218,
              4.439483, 53.942310, 2.080931],
  "y_mean": 319.70989,
  "y_scale": 53.93774,
  "feature_order": ["hour_sin", "hour_cos", ...]
}
```

Listing B.2: Model/scalers.json (abbreviated)

B.3 ESP32 Firmware (IOTcode.ino)

```
#include <Wire.h>
#include <Adafruit_BMP085.h>

Adafruit_BMP085 bmp;

void setup() {
  Serial.begin(115200);
  delay(3000);
  Wire.begin(21, 22);  // SDA=21, SCL=22
  if (!bmp.begin()) {
    Serial.println("BMP180 NOT FOUND!");
    while (1);
  }
  Serial.println("BMP180 FOUND!");
}

void loop() {
  Serial.print("Temperature : ");
  Serial.print(bmp.readTemperature());
  Serial.println(" C");
  Serial.print("Pressure    : ");
  Serial.print(bmp.readPressure() / 100.0);
  Serial.println(" hPa");
  Serial.print("Altitude    : ");
  Serial.print(bmp.readAltitude());
  Serial.println(" m");
  delay(2000);
}
```

Listing B.3: IOTcode/IOTcode.ino – BMP180 sensor sketch

Appendix C

Package Dependencies

Table C.1: Production NPM Dependencies

Package	Version	Purpose
next	16.2.10	Full-stack React framework
react	19.2.4	UI library
react-dom	19.2.4	React DOM renderer
typescript	^5	Type safety
tailwindcss	^4	Utility-first CSS
@tensorflow/tfjs	^4.22.0	Browser ML inference
@ducanh2912/next-pwa	^10.2.9	PWA service worker
recharts	^3.9.2	Time-series charts
maplibre-gl	^5.24.0	WebGL map rendering
react-map-gl	^8.1.1	React wrapper for MapLibre
framer-motion	^12.42.2	UI animations
gsap	^3.15.0	Scroll animations (landing)
lenis	^1.3.25	Smooth scroll
jspdf	^4.2.1	PDF generation
jspdf-autotable	^5.0.8	PDF table formatting
lucide-react	^1.23.0	Icon library
axios	^1.7.9	HTTP client
date-fns	^4.1.0	Date formatting utilities
@tanstack/react-query	^5.66.0	Data fetching cache
shadcn	^4.13.0	UI component system

Appendix D

Glossary and Acronyms

API	Application Programming Interface
BMP180	Bosch BMP180 digital pressure sensor (I^2C)
BiLSTM	Bidirectional Long Short-Term Memory neural network
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network
Conv1D	One-dimensional Convolutional Layer
DHT11	Digital Humidity and Temperature sensor
ESP32	Espressif Systems ESP32 dual-core Wi-Fi + Bluetooth SoC
GSAP	GreenSock Animation Platform
HC-SR04	HC-SR04 ultrasonic distance sensor
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
ISR	Incremental Static Regeneration (Next.js caching)
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory recurrent neural network
MAE	Mean Absolute Error
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
MSE	Mean Squared Error
PWA	Progressive Web Application

REST	Representational State Transfer
SSR	Server-Side Rendering
TF.js	TensorFlow.js
TIH	Technology Innovation Hub
TTL	Time To Live (cache duration)
UI	User Interface
URL	Uniform Resource Locator
WebGL	Web Graphics Library (browser GPU API)